

// Авторско право (с) 2026 Теодор Ненков  
// Лицензирано съгласно PolyForm Noncommercial License 1.0.0.  
// Търговската употреба изисква отделен лиценз.  
// Вижте ЛИЦЕНЗА за подробности.  
// Европа, България

## Част I. MAGIXTRAL

(<https://magixtral.com>)

### I. Magixtral - Main Page

#### 1. Sidebar

- Страничната лента се намира в лявата част на главната страница. Тя съдържа отделни клетки с падащи менюта и бутони.
- Ако натиснете бутона ">>", страничната лента ще се разшири и ще бъде видима.
- Ако натиснете бутона "<<", страничната лента ще се свие и ще бъде скрита.
- Ако натиснете и задържите дясната граница на страничната лента "↔" ще може да я разширявате и свивате ръчно.

#### 1.1. System logo

- Ако натиснете върху клетката с логото на операционната система (MAGI), ще се отвори прозорецът „Относно“, който съдържа информация за нея.

#### 1.2. Insight

- Ако натиснете превключвателя, системният режим ще се промени, при което се включва или изключва само AGI векторният слой от клетката „Системни слоеве“. Ако превключвателят е в лява позиция, системата ще бъде в AI режим. Ако превключвателят е дясна позиция, системата ще бъде в AGI режим.

#### 1.3. Temperature

- Ако промените позицията на плъзгача за температура, стойността на температурата на езиковия модел ще се промени. Чрез този плъзгач може да се регулира креативността.

#### 1.4. Engine model – езиков модел

- Ако натиснете на падащото меню с модела на двигателя, ще се покажат наличните езикови модели, които може да изберете за ползване. Базовият модел на двигателя на системата е „Mistral\_7B\_v0.3\_Q6\_K.gguf“.

//Могат да се добавят и други езикови модели към системата, като за целта е нужно да се добави името на езиковия модел във файла ollama\_list.json във файловия мениджър, чрез натискане на бутона за настройки в горен десен ъгъл, но за целта е нужно моделът да бъде предварително свален.

//Към настоящия момент се ползва оригиналния езиков модел на Mistral. Но освен него има друг езиков модел, който предстои да бъде качен в системата. На езиковия модел, който предстои да бъде качен, е извършена инструкционна фина настройка чрез квантувана нискорангова адаптация (QLoRA) по системата 4-4-2, като това ще се случи след завършване на всички необходими тестове и допълнителни настройки.

//Извършената фина настройка е на същия езиков модел: Mistral 7B v0.3. За целта е ползван набор от данни (когнитивен пакет), който е смесица от набори от данни (MoD – Mixture of Datasets) и съдържа информация от AGI слоя, който е базиран на четири двойки алгоритмични системи, които са взаимосвързани и от които са изградени три универсални шаблона за дедуктивен детерминизъм. Универсалните шаблони се отнасят за класификация на взаимодействия(за учене), структура на взаимодействия(за разбиране) и принципи на взаимодействия(за работа). Всеки от тези шаблони описва универсалните правила за възможни класификации, структури и принципи при извършване на приемо-предавателни процеси.

### **1.5. Personality**

- Ако натиснете на падащото меню с личности, ще се покажат наличните личности.
  - Ако натиснете върху личност от менюто, езиковият модел ще добави системната подкана за избраната личност към подканата за генериране на отговор.
  - Ако натиснете бутона "x" до някоя личност, можете да я изтриете.
  - Ако натиснете бутона "+" в клетката, можете да добавите своя нова личност към системата.
- //Чрез избиране на личност, езиковият модел започва да отговаря като тази личност.

### **1.6. Emotionality**

- Ако натиснете на падащото меню с емоционалности, ще се покажат наличните емоционални състояния (емоционалности).
  - Ако натиснете върху емоционалност от менюто, езиковият модел ще добави системната подкана за избраната емоционалност към подканата за генериране на отговор.
  - Ако натиснете бутона "x" до някоя емоционалност, можете да я изтриете.
  - Ако натиснете бутона "+" в клетката, можете да добавите своя нова емоционалност към системата.
- //Чрез избиране на емоционалност, езиковият модел започва да отговаря с това настрояние.

### **1.7. Value system**

- Ако натиснете на падащото меню на ценностната система, ще се покажат наличните качества на ценностната система.
  - Ако натиснете бутона "x" до някое качество на ценностната система, можете да го изтриете.
  - Ако натиснете бутона "+" в клетката, можете да добавите свое ново качество към системата.
  - Ако натиснете отметката (или отметки) в чекбокса вляво, качеството ще бъде маркирано или демаркирано. Можете да избирате множество качества от ценностната система.
- //Чрез избиране на качества от ценностната система, езиковият модел започва да ги прилага при поведението си в отговорите.
- //При създаване на нови личности, емоционални състояния и качества на ценностна система, е нужно системната подкана, да бъде кратка, ясна и непротиворечива, за да бъдат прилагани успешно от езиковия модел.

### **1.8. Documents**

- Ако натиснете на падащото меню с документи, ще се покажат наличните документи.
- Ако натиснете бутона "x" до някой документ, можете да го изтриете.
- Ако натиснете бутона "импорт" в клетката, можете да добавите .docx, .txt .json документи от компютъра си и да използвате техния контекст.

- Ако натиснете отметката (или отметки) в чекбокса вляво, документът ще бъде маркиран или демаркиран. Можете да избирате множество документи, като контекст само на тези с отметка ще бъде ползван от езиковия модел при генериране на отговор.

//Ако натиснете бутон "+", ще се появи прозорец с полета за писане и бутони "Cancel" и "Save", от които може да бъде затворен прозорецът или записан нов .json файл за personality, emotionality или value system.

## 1.9. System layers

- Ако натиснете на падащото меню със слоеве, можете да изберете векторните слоеве, които да използвате в системата (от чекбоксовете).

//AGI, Karma и други системни слоеве не могат да бъдат избирани.

//Може да се добавят векторни слоеве през файловия мениджър.

- Ако натиснете бутона "Dharma Wheel", ще се отвори прозорец, в който можете да видите информация за общия брой промптове на системата в активен режим (включително и тези от изтрити минали сесии), жизнения цикъл на системата, както и други данни.

## 1.10. Midextral

- Ако натиснете на клетката "midexXtral", ще отидете на midextral.com.

## 2. System page top toolbar „1“

### 2.1. Tabs (portals)

- Ако натиснете бутона "+" до таба, можете да добавите нов таб (портал).

- Ако натиснете бутона "x" в таба, можете да премахнете таб (портал).

- Ако натиснете и задържите таба (портала), можете да го движите и премествате с плъзгане и пускане.

## 3. System page top toolbar „2“

- Ако натиснете бутона "<", ще преминете към предишната сесия в историята си.

- Ако натиснете бутона ">", ще преминете към следващата сесия в историята си.

//Тези бутони могат да се използват само в активни режими, тъй като няма история в пасивните режими.

- Ако натиснете бутона "+ New chat", ще създадете нова сесия.

- Ако натиснете върху "Session bar", ще се покаже падащо меню със списък на сесиите.

- Ако натиснете на "Search bar", можете да търсите сесии по име.

- Ако натиснете върху сесия в падащото меню, ще я отворите в чата.

//Всяка сесия се номерира при създаването си.

- Ако натиснете бутона "x" на сесия, можете да я изтриете.

- Ако натиснете бутона "rename" на сесия, можете да я преименувате.

- Ако натиснете (маркирате) на чекбоксовете за сесия, можете да добавите контекста на маркираната сесия към текущата сесия, по която работите. Ако го махнете, ще премахнете контекста от текущата сесия.

//Контекстът на маркирана сесия се премахва автоматично при излизане от текущата сесия. Това е полезно за започване на нов чат или за добавяне на контекст от друга сесия по време на текуща сесия.

- Ако натиснете на бутона "WEB", можете да включите или изключите уеб търсенето.

//Ако не сте избрали serper key за Google, уеб бутонът няма да е активен.

- Ако натиснете бутона "Serper key", от бутоните в падащото меню, което ще се появи, можете да изберете, добавите или премахнете serper key.
- Ако натиснете бутона "Export", ще се появи падащо меню и можете да изберете да експортирате текущата сесия в: .pdf, .docx, .txt, .jsonl (lmft), .jsonl (ift).  
//LMFT – експортиране като набор от данни за Language Model Fine-Tuning;  
//IFT – експортиране като набор от данни за Instruction Fine-Tuning;
- Ако натиснете бутона "Day/Night", можете да превключите темата на системата от светлата охрена тема на тъмната сива тема.

#### 4. Chat workspace

- Ако поставите мишката върху подкана в чата, над подканата ще се появят бутоните "copy", "Edit and regen" и "Delete".
- Ако натиснете бутона "Copy", ще копирате подканата и тя ще бъде добавена в клипборда на приложението Docstral.
- Ако натиснете бутона "Edit and regen", ще се появи подканата в полето за въвеждане на подкана, а в чата подканата и отговорът под нея ще избледнеят. В тази ситуация имате две опции: При първата опция, можете допишете подканата в полето долу, след което при натискане на бутона за изпращане или "enter", избледнелите подкана и отговор ще изчезнат и ще се появи новата подкана и ще се генерира нов отговор за нея в чата. При втората опция, можете да изтриете подканата в полето долу, при което се отказва извършването на регенериране на отговора и изчезва избледняването на подканата и отговора. Допълнително: ако при редактиране на подканата, искате да я премахнете напълно без това да отказва извършването на регенериране, можете да натиснете бутона вдясно на полето за подкани "Clear text".
- Ако натиснете бутона "Delete", ще изтриете подканата и отговора.
- Ако поставите мишката върху подкана в чата, над подканата ще се появят бутоните "copy", "Edit and regen" и "Delete".
- Ако натиснете бутона "Copy", ще копирате отговора и той ще бъде добавен в клипборда на Docstral.
- Ако натиснете бутона "Edit", можете да редактирате отговора, да добавите или премахнете неща в него и редактираният отговор ще бъде използван като контекст от езиковия модел, старият отговор ще бъде забравен.
- Ако натиснете бутона "Cancel" на полето за редактиране на отговора, можете да излезете от режима на редактиране.
- Ако натиснете "Save", на полето за редактиране на отговора можете да запазите редактирания отговор и да излезете от режима на редактиране.
- Ако натиснете и задържите в ъгъла на полето за редактиране на отговора (в режим на редактиране), можете да промените размера на полето.
- Ако натиснете бутона "Retry", ще изтриете текущия отговор и на негово място ще се генерира нов като се ползва същата подкана.
- Ако са ползвани бутоните "Edit", "Retry", "Erase", "Regen Selected" или "Expand Selected", когато поставиш мишката върху отговора, ще се покаже допълнителен бутон "Undo".
- Ако натиснете бутона "Undo", ще върнете състоянието на отговора преди извършената промяна и ще се покаже нов допълнителен бутон "Redo".
- Ако натиснете бутона "Redo", ще върнете обратно променения отговор.

## 5. Chat container

- Ако натиснете върху "Chat container", можете да напишете подкана.
- Ако натиснете бутона "Send" отдясно в полето за чат или натиснете "Enter", езиковият модел ще генерира отговор.
- Ако натиснете бутона "Send" или натиснете "Enter", бутонът за изпращане ще се превърне в бутон "Stop".
- Ако натиснете бутона "Stop", ще спрете генерирането на отговори и в отговора ще се появи знак "Stopped". Когато въведете следваща подкана, текстът "Stopped" ще стане сив.
- Ако натиснете бутона "Appstrale" отляво в полето за чат, ще се появи падащо меню с бутоните за отваряне на приложенията на операционната система: "Webztral", "Docstral", "Imvistrall", "File manager" и връзка към външен сайт "Ask a colleague".
- Ако натиснете бутона "Webztral", ще отворите вътрешния системен браузър.
- Ако натиснете бутона "Docstral", ще отворите вътрешния текстов редактор на системата.
- Ако натиснете бутона "Imvistrall", ще отворите вътрешното медийно пространство на системата.
- Ако натиснете бутона "File manager", ще отворите вътрешния системен файлов мениджър.
- Ако натиснете бутона "Ask a colleague", ще отворите "Mistral Le Chat" в реалния си външен браузър.
- Ако натиснете върху "Chat container", бутоните "Paste", "Continue" и "Explain" ще се покажат.
- Ако натиснете върху друго поле за писане в системата, само бутонът "Paste" ще се покаже.
- Ако преместите мишката настрани от бутоните, те ще избледняват и накрая ще изчезнат.
- Ако кликнете отстрани на екрана, бутоните ще изчезнат.
- Ако натиснете бутона "Paste", ще поставите копиран текст в текстовото поле.
- Ако натиснете бутона "Continue", езиковият модел ще генерира продължение на предишния отговор, като ползва системна подкана за продължаване на генерирането по същата тема.
- Ако натиснете бутона "Explain", езиковият модел ще генерира обяснение на предишния отговор, като ползва системна подкана за това.

Бутони при маркиране на текст на основната страница:

- Ако маркирате текст от подкана или от отговор, над него ще се появят бутони.
- Ако натиснете бутона "Copy", избраният текст ще бъде копиран и поставен в клипборда на Docstral.
- Ако натиснете бутона "Ask here", избраният текст ще бъде поставен в "Chat container" на текущия таб (портал) с опция да допишете подканата.
- Ако натиснете бутона "Ask elsewhere", избраният текст ще бъде поставен в "Chat container" в друг таб (портал) с опция да допишете подканата.
- Ако натиснете бутона "Summary", езиковият модел ще генерира обобщение на избрания текст.
- Ако натиснете бутона "Explain", езиковият модел ще генерира обяснение на избрания текст.
- Ако натиснете бутона "v", ще се появи падащо меню с допълнителни бутони.
- Ако натиснете бутона "^", можете да скриете падащото меню с бутони.
- Ако натиснете бутона "Find inside", избраният текст ще бъде търсен в Google във вътрешния браузър (Webztral), който ще се отвори.
- Ако натиснете бутона "Find outside", избраният текст ще бъде търсен в Google във външния браузър (истинския браузър), който ще отвори нов раздел.

- Ако натиснете бутона "Mark as code", избраният текст ще бъде поставен в блок за код в чата.
- Ако маркирате код в блок за код, можете да натиснете бутона "Mark as text", което ще извади кода от блока и ще го постави като текст в чата. Ако извадите код от средата на блок за код, ще се появят два блока за код и изваденият текст ще бъде между тях.
- Ако натиснете бутона "Erase", избраният текст ще бъде премахнат от подканата или отговора и ще бъде забравен от системата.
- Ако натиснете бутона "Link it", ще се появи прозорец с поле за писане и бутоните "Save" и "Cancel". Ако въведете линк и натиснете "Enter" или "Save", избраният текст ще се превърне в хипервръзка, а ако натиснете "Cancel" ще прекратите създаването на връзка.
- Ако натиснете на "Regen selected", избраният текст ще изчезне от отговора и ще се появи в "chat container" като подкана, след което има две опции. При първата опция, ако се въведе допълнение към тази подкана, езиковият модел ще генерира отговор и ще замени само избрания текст. При втората опция, ако се изтрие взетата подкана от полето за писане, тя ще се появи отново в текста и процесът по регенериране ще бъде прекратен.
- Ако натиснете "Expand selected", избраният текст ще изчезне от отговора и ще се появи в "chat container" като подкана, след което има две опции. При първата опция, ако се въведе допълнение към тази подкана, езиковият модел ще генерира разширен отговор и ще замени само избрания текст. При втората опция, ако се изтрие взетата подкана от полето за писане, тя ще се появи отново в текста и процесът по разширяване ще бъде прекратен.
- Ако натиснете бутона "Astral projection", избраният текст ще бъде поставен в нова клетка в прозореца на "Astral projection".
- Ако преместите мишката настрани от падащото меню с бутони, то ще избледнее и накрая ще изчезне. Ако кликнете отстрани на екрана, падащото меню с бутони ще изчезне.

## 6. Links:

- Ако натиснете на линк в чата, ще се появи падащо меню с бутони.
- Ако натиснете бутона "Open inside", ще отворите линка в Webztral.
- Ако натиснете "Open outside", ще отворите линка в истинския браузър.
- Ако натиснете "Edit link", можете да редактирате линка от прозореца, който се появява.
- Ако натиснете "Unlink", ще премахнете хипервръзката.

## 7. Session navigation bar

- Ако натиснете бутона "<<", ще отидете на първата подкана в текущата сесия.
- Ако натиснете бутона "<", ще отидете на предишната подкана.
- Ако натиснете бутона ">", ще отидете на следващата подкана.
- Ако натиснете бутона ">>", ще отидете на последната подкана.
- Ако напишете число в полето за писане на навигационната лента и натиснете "Enter", ще отидете на подканата с въведения номер. Можете да видите общия брой подкани в текущата сесия и подканата на която се намирате в момента.

//Вдясно на всяка подкана има число за поредния номер на подканата в текущата сесия.

## 8. Addons

- Ако натиснете бутона "Clear text", ще изтриете наличен текст в "Chat container" и полето за чат ще бъде натиснато автоматично, като по този начин можете да директно да напишете нова подкана.
- Ако натиснете бутона "Astral projection", ще отворите прозореца на астралната проекция.

## 9. Code block

- Ако езиковият модел генерира код, той ще се появи в чата в "code block".
- Генерираният код в "code block" е "Syntax highlighted".
- Всеки "Code block" има номерация на редовете като при IDE.
- Всеки "Code block" съдържа информация за броя редове и броя на символите в долната лява част.
- Всеки "Code block" има бутони:
  - Ако натиснете бутона "Astral" в долната част на блока за код, ще добавите кода от блока към нова клетка в прозореца на "Astral projection".
  - Ако натиснете бутона "Copy" в горната част на блока за код, ще копирате кода от блока, но той няма да бъде добавен в клипборда на Docstral.
  - Ако натиснете бутона "Unlock" в горната част на блока за код, ще можете да редактирате кода ръчно и бутонът ще се промени на "Lock".
  - Ако натиснете бутона "Lock" в горната част на блока за код, ще запазите редактирания код и бутонът ще се промени на "Unlock".
  - Ако натиснете бутона "Export", ще се появи падащо меню с файлови формати. Ако натиснете бутон: .txt, .json, .bat, .docx, .py, .html, .js, .css, .c, .cpp, ще изтеглите файл с този формат, който съдържа кода.

## 10. System page bottom toolbar „1“

- Ако натиснете бутона "Fully passive", ще активирате този работен режим в отворения таб.
  - Ако натиснете бутона "Partially passive", ще активирате този работен режим в отворения таб.
  - Ако натиснете бутона "Fully active", ще активирате този работен режим в отворения таб.
  - Ако натиснете бутона "Partially active", ще активирате този работен режим в отворения таб.
- //Чрез промяна на работните режими се променя начинът на работа на системата.

## II. Magixtral - Appstrale (applications) and addons

### 1. WEBZTRAL

#### 1.1. Webztral window topbar:

- Ако натиснете бутона "Minimize", Webztral ще се скрие и на основната страница ще се появи Webztral bubble (балонче).
  - Ако натиснете върху балончето на Webztral, ще се отвори Webztral.
  - Ако натиснете бутона "Maximize" или ако натиснете два пъти върху Webztral window topbar, докато е в режим на цял екран, ще превключите на режим прозорец.
  - Ако натиснете и задържите границите на прозореца Webztral, можете да промените размера на прозореца.
  - Ако натиснете два пъти на Webztral window topbar, докато е в режим прозорец, ще превключите в режим на цял екран, а ако е обратното ще превключите в режим прозорец.
  - Ако натиснете бутона "X", ще затворите Webztral.
- Ако натиснете и задържите на Webztral window topbar, можете да местите прозореца му.
- //Същата логика за преоразмеряване, отваряне, затваряне, минимизиране, максимизиране и местене на прозорците, се ползва и при прозорците на "About", "Webztral", "Docstral", "File manager" и "Astral projection".

## 1.2. Webztral top toolbar and bookmarks bar:

- Ако натиснете бутона "←", ще отидете на предишната страница от историята.
- Ако натиснете бутона "→", ще отидете на следващата страница от историята.
- Ако натиснете бутона "Home", ще отидете на google.com.
- Ако натиснете върху "Address bar", можете да въведете линк и да го отворите като натиснете на "Enter" или бутона "Go →".
- Ако натиснете бутона "Add a bookmark", ще се появи прозорец с полета за писане и бутоните "Save" и "Cancel", от където можете да добавите отметка или да прекратите добавянето.
- Ако натиснете десен бутон на отметка, ще се появи прозорец с полета за писане и бутоните "Delete", "Cancel" и "Save", от където можете да премахнете или редактирате отметка.
- Ако натиснете върху отметка, ще я отворите вътре в Webztral.
- Ако натиснете бутона "Outside", ще отворите настоящата страница в нов таб на истинския браузър.
- Ако натиснете върху бутона "History", ще се появи падащо меню с последните посетени сайтове. От тук може да видите историята на сърфирането за текущата сесия.
- Ако натиснете "Delete", можете да изтриете страници от историята.
- Ако натиснете "Clear all", можете да изчистите цялата история за тази сесия.
- Ако натиснете бутона "Export", ще се появи падащо меню с бутони за експортиране.
- Ако натиснете бутона "Selected to png", можете да изберете зона от видимия екран чрез маркиране, след което се появяват бутоните "Save" и "X" и можете да я запазите като png или да откажете.
- Ако натиснете бутона "Visible to png", можете да изтеглите видимата страница като png.
- Ако натиснете бутона "Full to png", можете да изтеглите цялата страница като png.

## 2. Docstral

### -Docstral window bar:

//Ползва се същата логика за преоразмеряване, отваряне, затваряне, минимизиране, максимизиране и местене като на останалите прозорци, но има допълнителни бутони в тази лента.

- Ако натиснете бутона "Zen Mode", ще скриете горната лента с инструменти „1“, горната лента „2“, долната лента „2“ и долната лента „1“, което ви позволява да се фокусирате по-добре в работата като премахнете излишния товар за очите.
- Ако натиснете бутона "Planchette Mode", ще отворите основната част на Docstral, където можете да работите с текстови файлове.
- Ако натиснете бутона "PDF mode", ще отворите разширение на Docstral, където можете да управлявате .pdf, .jpg .png файлове.
- Ако натиснете бутона "QR generator", ще отворите разширение на Docstral, където можете да създавате QR кодове.

### 2.1. Docstral – Planchette Mode

// "Planchette mode" има две горни и две долни ленти и лента на прозореца.

// "Pdf mode" и "QR generator" имат само лента на прозореца.

### 2.1.1. Docstral top toolbar „1“

- Ако натиснете бутона "<", ще отворите предишния документ от списъка с документи в "document bar".
- Ако натиснете бутона ">", ще отворите следващия документ от списъка с документи в "document bar".
- Ако натиснете бутона "Import", можете да импортирате текстови файлове в .docx, .txt, .jdoc.json формат.
- Ако натиснете бутона "+", можете да създадете нов .jdoc.json текстов файл (документ).
- Ако натиснете бутона "Document bar", ще се появи падащо меню със списък с всички налични документи от папката `planchette_mode` във файловия мениджър, като вдясно на всеки документ има бутон за преименуване и бутон за изтриване на документа.
- Ако натиснете на документ от списъка, ще го отворите.
- Ако натиснете бутона "Rename", можете да преименувате документа.
- Ако натиснете бутона "Delete", можете да изтриете документа.
- Ако натиснете бутона "-", можете да изтриете документа, който е отворен.
- Ако натиснете бутона "Export", ще се появи падащо меню с 5 бутона за експортиране във формат .docx, .pdf, .txt, .jdoc.json, .png.
- Ако натиснете бутона "↶", ще върнете предишното състояние на документа (Ctrl + Z).
- Ако натиснете бутона "↷", ще върнете настоящето състояние на документа (Ctrl + Y).

### 2.1.2. Docstral top toolbar „2“

- Ако натиснете бутона "-", ще намалите мащаба на документа.
- В полето на лентата за мащабиране можете, да видите мащаба. Ако въведете число (от 0 до 250 %) в нейното поле за писане и натиснете "Enter", можете да промените мащаба на документа.
- Ако натиснете бутона "+", можете да повишите мащаба на документа.
- Ако натиснете бутона "Find and Replace", вляво ще се отвори страничната лента за търсене и замяна в документа.
- Ако натиснете бутона "Clipboard", вляво ще се отвори страничната лента на клипборда.
- Ако натиснете бутона "Bold", можете да удебелите шрифта на маркиран текст.
- Ако натиснете бутона "Italic", можете да наклоните шрифта на маркиран текст или да пишете с наклонен шрифт на реда.
- Ако натиснете бутона "Underline", можете да подчертаете маркиран текст или да пишете подчертан текст на реда.
- Ако натиснете бутона "Strikethrough", можете да зачеркнете маркиран текст или да пишете зачеркант текст на реда.
- Ако натиснете бутона "Superscript", можете да направите маркиран текст като горен индекс или да пишете с горен индекс на реда.
- Ако натиснете бутона "Subscript", можете да направите маркиран текст като долен индекс или да пишете с долен индекс на реда.
- Ако натиснете бутона "Clear Formatting", можете да премахнете форматирането на маркиран текст, като това ще премахне всички: bold, italic, underline, strikethrough, superscript, subscript, font, font colour, size, Heading 1, 2, 3, 4, Title, Quote, Align left, center, right и justify. Маркираният текст ще стане TEXT layer, Align left, в черен цвят, Times New Roman, 12pt.
- Ако натиснете бутона "Astral projection", можете да добавите маркиран текст от документа към прозореца на астралната проекция.

- Ако натиснете бутона "Eagle View", вдясно ще се отвори страничната лента на орловия поглед.
- Ако натиснете бутона "Tables", вляво ще се отвори страничната лента за таблиците.
- Ако натиснете бутона "Font", ще се появи падащо меню с шрифтове, от където може да се промени шрифта на маркиран текст.
- Ако натиснете или задържите бутона "v", можете да понижите размера на шрифта на маркиран текст или ред.
- Ако натиснете или задържите бутона "^", можете да повишите размера на шрифта на маркиран текст или ред.
- Ако натиснете бутона "Text colour", ще се появи прозорец със стандартни цветове и опция за custom RGB colour, от където може да се промени цвета на маркиран текст или ред.
- В клетката "Current font and size", можете да видите шрифта и размера на маркиран текст. Ако маркираният текст е еднороден, ще се покаже примерно: "Times New Roman, 12pt", но ако в маркирания текст има части, които са с различен размер или шрифт, в клетката ще се покаже: "Mixture of Fonts".

### **2.1.3. Docstral bottom toolbar „2“**

- Ако натиснете бутона "Layers", вляво ще се отвори страничната лента за слоевете.
- Ако натиснете бутона "H1", можете да направите маркиран текст като слой Heading 1.
- Ако натиснете бутона "H2", можете да направите маркиран текст като слой Heading 2.
- Ако натиснете бутона "H3", можете да направите маркиран текст като слой Heading 3.
- Ако натиснете бутона "H4", можете да направите маркиран текст като слой Heading 4.
- Ако натиснете бутона "TTL", можете да направите маркиран текст като слой Title.
- Ако натиснете бутона "Q", можете да направите маркиран текст като слой Quote.
- Ако натиснете бутона "TXT", можете да направите маркиран текст като слой Text (base).
- Ако натиснете бутона "Align Left", можете да подравните текст вляво на страницата.
- Ако натиснете бутона "Center", можете да центрирате текст по средата на страницата.
- Ако натиснете бутона "Align Right", можете да подравните текст вдясно на страницата.
- Ако натиснете бутона "Justify", можете да подравните текст едновременно вляво и вдясно на страницата.
- Ако натиснете бутона "Lines", вляво ще се отвори страничната лента за редовете.
- В клетката "Current style", можете да видите стила(слоя) на маркиран текст.

### **2.1.4. Docstral bottom toolbar „1“**

- Ако натиснете бутона "<<", ще отидете на първия ред в текущия документ.
- Ако натиснете бутона "<", ще отидете на предишния ред.
- Ако натиснете бутона ">", ще отидете на следващия ред.
- Ако натиснете бутона ">>", ще отидете на последния ред.
- Ако напишете число в полето за писане на навигационната лента на Docstral и натиснете "Enter", ще отидете на реда с въведения номер. Можете да видите общия брой редове в текущия документ и реда на който се намирате в момента.
- Ако въведете подкана в интегрираното поле за писане и натиснете "Enter", езиковият модел ще генерира отговор в документа, където се намира каретът(курсора) в момента.
- Ако натиснете бутона "Content", вляво ще се отвори страничната лента за съдържанието.
- Ако натиснете бутона "Info", вляво ще се отвори страничната лента за информация.
- Ако натиснете бутона "Print", можете да принтирате документа.

## **-Docstral sidebars:**

### **2.1.5. Docstral Placnhette mode sidebar (Home screen)**

- Ако натиснете бутона "X", можете да затворите страничната лента.
- Ако натиснете бутона "New document", можете да създадете нов .jdoc.json текстов файл (документ).
- Ако натиснете на някой от документите в списъка "recent files", можете да го отворите.

### **2.1.6. Find and Replace Sidebar**

- Ако натиснете текстовото поле "Find what", можете да въведете текст за търсене.
  - Ако натиснете текстовото поле "Replace with", можете да въведете текст за замяна.
  - Ако натиснете бутона "Find", ще търсите въведения текст от полето "Find what".
  - Ако натиснете бутона "Replace all", ще замените текста на всички намерени резултати от търсенето с въведения текст от полето "Replace with".
  - Ако натиснете бутона "Replace Selected", ще замените текста само на избраните намерени резултати от търсенето, които са отбелязани с чекбокс в списъка с резултати, с въведения текст от полето "Replace with".
  - Ако натиснете бутона "←", ще отидете на предишния резултат от списъка с намерени.
  - Ако натиснете бутона "→", ще отидете на следващия резултат от списъка с намерени.
  - Ако натиснете основния чекбокс "Select all" на списъка с намерени резултати, можете да маркирате или да демаркирате всички резултати.
- //Вдясно на чекбокса "Select all", можете да видите общия брой намерени резултати и броя на маркираните с чекбокс резултати.
- Ако затворите страничната лента ще изчезне оцветяването на фона на текста (намерените резултати) в документа.

### **2.1.7. Clipboard Sidebar**

- Ако натиснете бутона "Clear all", ще изчистите всички клетки с копиран текст в клипборда.
- Ако натиснете бутона "Copy", ще копирате текста от клетката.
- Ако натиснете бутона "Regen", ще се появи поле за писане, при което имате две опции: При първата опция, можете да въведете подкана и да натиснете "Enter", след което в документа ще се генерира отговор. При втората опция, можете да натиснете повторно бутона "Regen", за да се скрие полето за писане.
- Ако натиснете на бутона "Delete", ще изтриете клетката с текста.

### **2.1.8. Eagle View Sidebar**

- Ако поставите мишката върху страничната лента на "Eagle view", ще се появи малък прозорец вдясно, в който се виждат около 50 реда от документа в малък мащаб.
  - Ако местите мишката вертикално (нагоре и надолу) в страничната лента, можете да наблюдавате целия документ от високо.
  - Ако спрете да местите мишката и натиснете страничната лента, ще замразите изгледа на прозореца за визуализация.
  - Ако поставите мишката върху част от прозореца за визуализация, ще се появи правоъгълен прозорец (лупа), в който се виждат около 7 реда от документа в по-голям мащаб в реално време, като по този начин текстът се чете лесно.
  - Ако натиснете прозореца (лупа), ще замразите изгледа му.
- //При замразяване на изгледа в прозореца и в лупата, можете да скролвате и да работите на друго място в документа, докато наблюдавате текста в лупата.

### 2.1.9. Tables Sidebar

- Ако въведете числа в полетата за писане "Rows" и "Columns", след което натиснете бутона "Insert table", ще създадете нова таблица с толкова редове и колони на реда, който е натиснат последно.
- Ако натиснете бутона "↑", ще добавите нов ред над маркирана клетка в таблицата.
- Ако натиснете бутона "↓", ще добавите нов ред под маркирана клетка в таблицата.
- Ако натиснете бутона "←", ще добавите нова колона вляво на маркирана клетка в таблицата.
- Ако натиснете бутона "→", ще добавите нова колона вдясно на маркирана клетка в таблицата.
- Ако натиснете бутона "Delete row", ще изтриете реда на маркирана клетка от таблицата.
- Ако натиснете бутона "Delete column", ще изтриете колоната на маркирана клетка от таблицата.
- Ако натиснете бутона "Delete table", ще изтриете таблицата, в която е маркирана клетка.
- В полето "Marked cells for context", можете да видите клетките, които са маркирани в момента и от които езиковият модел може да ползва като контекст за отговор (например A1, B2).
- В полето "Enter a prompt here", може да въведете подкана и ако натиснете "Enter" или бутона "Generate answer", ще се генерира отговор с контекста на маркираните клетки, на реда под таблицата.
- Ако натиснете бутона "Explain data", ще се генерира отговор от езиковия модел, на реда под таблицата, който съдържа обяснение на цялата таблица.
- Ако натиснете "Export to python pandas", ще се генерира python код на реда под таблицата. Чрез генерирания python script, може да се създаде таблица в .xlsx формат при изпълнение през терминал.
- Ако натиснете бутона „Ruler”, можете да измервате дължини в милиметри в документа.

### 2.1.10. Layers Sidebar

- Ако натиснете чекбокса "Select all layers", можете да маркирате или да демаркирате всички слоеве от списъка.
  - Ако натиснете бутона "Toggle visibility for all", можете да скриете съдържанието на всички слоеве от документа.
- //Скриването на слоеве не изтрива съдържанието, а го прави невидимо, като пренарежда редовете на целия документ
- В страничната лента има общо 15 слоя: Heading 1 All, Heading 1 Names, Heading 1 Content, Heading 2 All, Heading 2 Names, Heading 2 Content, Heading 3 All, Heading 3 Names, Heading 3 Content, Heading 4 All, Heading 4 Names, Heading 4 Content, Title, Quotes и Content (text and tables). Вдясно на всеки слой има бутон "Ask Magi for this layer" и "Toggle visibility", а вляво има чекбокс.
- //All – имена на заглавия, текст и таблици, Names – само имена на заглавия и Content – само текст и таблци за избрания "Heading".
- Ако натиснете бутона "Ask Magi for this layer", ще се появи поле за писане, в което можете да въведете подкана и ако натиснете "Enter", ще се генерира отговор на реда, който е натиснат последно, като в него ще се вземе като контекст само текстът от избрания слой.
  - Ако натиснете бутона "Toggle visibility", можете да скривате текста на този слой от документа или да го възстановите обратно.

- Ако натиснете бутона "Layer actions", ще се появи падащо меню с бутони(опции) с три раздела: Раздел "Create outside – New .jdoc", в който са бутоните "From selected layers" и "From all layers", Раздел "Create inside – New version", в който са бутоните "From selected layers" и "From all layers", Раздел "Other actions", в който са бутоните "Export selected to .txt file" и "Remove selected from here".

- Ако натиснете бутона "From selected layers" (в раздел create outside), ще създадете нов .jdoc.json документ, който съдържа текста само на избраните слоеве.

- Ако натиснете бутона "From all layers" (в раздел create outside), ще създадете нов .jdoc.json документ, който съдържа текста на всички слоеве.

- Ако натиснете бутона "From selected layers" (в раздел create inside), ще създадете нова версия на документа в същия .jdoc.json документ, в който работите, като новата версия ще съдържа текста само на избраните слоеве.

- Ако натиснете бутона "From all layers" (в раздел create inside), ще създадете нова версия на документа в същия .jdoc.json документ, в който работите, като новата версия ще съдържа текста на всички слоеве.

//При създаването на нов документ или нова версия, се появява прозорец в който трябва да се въведе тяхното име.

//След създаване на нова версия на документа, тя се появява в списъка "Document versions" в страничната лента под списъка със слоеве. Може да се превключват версиите на документа чрез натискане върху клетката на версията в списъка. По този начин един документ може да има отделна версия за различен език или документ да има свързани към него документи под формата на различни версии. Тоест може да се създаде нова версия от един слой, но не е нужно текстът да е същия като в основната версия.

- Ако натиснете бутона "Export selected to .txt", можете да свалите текста на избраните слоеве в текстов файл.

- Ако натиснете бутона "Remove selected from here", ще изтриете текста на избраните слоеве от документа и редовете ще се пренаредят.

//В Docstral всеки ред е параграф и няма отстъпи.

//Версиите в Docstral наподобяват на "Git" за писатели.

### **2.1.11. Lines Sidebar**

- Ако въведете число (брой редове) в полето за писане, след което от чекбоксовете маркирате номерата на редове и натиснете бутона "Add before", ще добавите посочения брой нови редове преди избраните.

- Ако въведете число (брой редове) в полето за писане, след което от чекбоксовете маркирате номерата на редове и натиснете бутона "Add after", ще добавите посочения брой нови редове след избраните.

- Ако натиснете на бутона "Delete all", ще изтриете всички редове и съдържанието им.

- Ако натиснете на бутона "Delete selected" и маркирате редове от списъка чрез чекбоксовете, ще изтриете избраните редове и съдържанието им.

- Ако в полето за писане "Remove specific" въведете номера на редове (например 1, 2, 5-10) и натиснете бутона "Delete", ще изтриете тези редове и съдържанието им.

- Ако натиснете чекбокса "All", можете да маркирате или демаркирате всички редове от списъка в страничната лента.

- Ако натиснете чекбокса "Even", можете да маркирате или демаркирате всички четни редове от списъка в страничната лента.

- Ако натиснете чекбокса "Odd", можете да маркирате или демаркирате всички нечетни редове от списъка в страничната лента.
- Всеки ред от списъка има бутон "Delete". Ако натиснете бутона "Delete", ще изтриете реда и съдържанието му.
- Всеки ред в списъка е като клетка, която съдържа чекбокс, пореден номер на реда, част от текста на всеки отделен ред, слоя на текста на реда и бутона за изтриване.

### 2.1.12. Content Sidebar

- Ако въведете номер на ред от документа в полето за писане "Add before line", след което изберете елементите на съдържанието от списъка чрез чекбоксовете и натиснете на бутона "Create Table of Contents", ще се появи съдържанието на документа, преди посочения ред в полето за писане.
- Списъкът за създаване на съдържание включва 5 неща: Titles, Headings 1, 2, 3 и 4.

### 2.1.13. Info Sidebar

- Страничната лента за информация съдържа две клетки с данни, които се променят постоянно при работа.
- Списъкът "Structure counts" показва данни за брой: Title blocks, Headings 1, Headings 2, Headings 3, Headings 4 и Quotes.
- Списъкът "Word Distribution" показва данни за брой: Total words, Words in Titles, Words in Headings 1, Words in Headings 2, Words in Headings 3, Words in Headings 4 и Words in Quotes.

### 2.1.14. Docstral workspace:

Бутони при маркиране на текст в Docstral:

- Ако маркирате текст в Docstral, под него ще се появи падащо меню с бутони.
- Ако натиснете бутона "Bold" в менюто с бутони, можете да удебелите шрифта на маркиран текст.
- Ако натиснете бутона "Italic" в менюто с бутони, можете да наклоните шрифта на маркиран текст или да пишете с наклонен шрифт на реда.
- Ако натиснете бутона "Underline" в менюто с бутони, можете да подчертаете маркиран текст или да пишете подчертан текст на реда.
- Ако натиснете бутона "Strikethrough" в менюто с бутони, можете да зачеркнете маркиран текст или да пишете зачеркант текст на реда.
- Ако натиснете бутона "Superscript" в менюто с бутони, можете да направите маркиран текст като горен индекс или да пишете с горен индекс на реда.
- Ако натиснете бутона "Subscript" в менюто с бутони, можете да направите маркиран текст като долен индекс или да пишете с долен индекс на реда.
- Ако натиснете бутона "Clear Formatting" в менюто с бутони, можете да премахнете форматирането на маркиран текст.
- Ако натиснете бутона "Astral projection" в менюто с бутони, маркираният текст ще бъде поставен в нова клетка в прозореца на "Astral projection".
- Ако натиснете бутона "Cut" в менюто, маркираният текст ще бъде изрязан и поставен в клипборда на Docstral.
- Ако натиснете бутона "Copy" в менюто, маркираният текст ще бъде копиран и поставен в клипборда на Docstral.
- Ако натиснете бутона "Paste" в менюто, на мястото на маркирания текст ще бъде поставен копиран текст.

- Ако натиснете бутона "Find Here" в менюто, ще се отвори страничната лента на "Find and Replace" и маркираният текст ще бъде поставен в полето за писане "Find what", след което ще се покажат резултати от търсенето в списъка на страничната лента.
- Ако натиснете бутона "Find Inside" в менюто, ще се отвори вътрешният браузър "Webztral" и маркираният текст ще бъде търсен в google.com
- Ако натиснете бутона "Find Outside" в менюто, ще се отвори нов таб на истинския браузър и маркираният текст ще бъде търсен в него.
- Ако натиснете на полето за писане в менюто, след което въведете подкана и натиснете "Enter", ще се генерира текст на мястото на маркирания текст.
- Ако преместите мишката настрани от падащото меню с бутони, то ще избледнее и накрая ще изчезне. Ако кликнете отстрани на екрана, падащото меню с бутони ще изчезне.

## 2.2. Docstral – PDF Mode

- Отляво има странична лента. Отдясно е работното поле.
- Ако в страничната лента натиснете полето "Drag and drop files" или преместите файл от компютъра си, можете да качите файлове в "Pdf mode". Качените файлове ще се появят в списъка с файлове в страничната лента вляво, а техните страници ще се появят в работното поле вдясно. Може да се качват няколко .pdf, .jpg, .png файлове, като в работното поле всички .pdf файлове се показват като отделни страници, които са номерирани и на които е изписано името на оригиналния .pdf документ.
- Ако натиснете бутона "X" вдясно на документ от списъка, ще го изтриете от списъка и ще премахнете неговите страници от работното поле.
- В полето "Selected" в страничната лента, можете да видите броя маркирани страници в работното поле.
- От чекбокса в страничната лента, можете да изберете дали .pdf документа, който ще бъде експортиран накрая, да има номера на страниците (Pagination).
- В полето за писане в страничната лента, можете да въведете текст, който да се показва като воден знак на всяка страница в експортирания документ.
- Ако натиснете бутона "Merge all to pdf", ще съедините всички страници от работното поле в един .pdf документ и той ще бъде изтеглен.
- Ако натиснете бутона "Merge selected to pdf", ще съедините само маркираните страници от работното поле в един .pdf документ и той ще бъде изтеглен.
- Ако натиснете бутона "Split all to pdf", ще изтеглите всички страници от работното поле като отделни .pdf файлове, които са събрани в един .zip файл.
- Ако натиснете бутона "Split selected to pdf", ще изтеглите маркираните страници от работното поле като отделни .pdf файлове, които са събрани в един .zip файл.
- Ако натиснете бутона "Split all to png", ще изтеглите всички страници от работното поле като отделни .png файлове, които са събрани в един .zip файл.
- Ако натиснете бутона "Split selected to png", ще изтеглите маркираните страници от работното поле като отделни .png файлове, които са събрани в един .zip файл.
- Ако натиснете и задържите върху работното поле, можете да маркирате страници чрез "Drag selection".
- Ако натиснете веднъж на немаркирана страница, ще я маркирате.
- Ако натиснете веднъж на маркирана страница, ще я демаркирате.
- Ако натиснете на страница и задържите, можете да преместите местоположението на тази страница на друго място в документа и при експортирането тя ще бъде на този пореден номер.

- Ако натиснете два пъти на страница, ще отворите .pdf визуализатор, в който може да скролвате през всички страници в работното поле, като един документ.
- Ако натиснете бутона "Zoom in" в .pdf визуализатора, ще повишите мащаба на документа.
- Ако натиснете бутона "Zoom out" в .pdf визуализатора, ще понижите мащаба на документа.
- Ако натиснете бутона "X" в .pdf визуализатора, ще излезете от визуализатора.
- Ако поставите мишката върху страница в работното поле, ще се появят бутоните "Rotate left", "Rotate right" и "Delete" над страницата.
- Ако натиснете бутона "Rotate left", ще завъртите страницата наляво (-90°).
- Ако натиснете бутона "Rotate right", ще завъртите страницата надясно (90°).
- //Можете да въртите всяка отделна страница на 360° и тя ще се визуализира по този начин в работното поле и в експортирания документ или снимка.
- Ако натиснете бутона "Delete", можете да изтриете страница от работното поле.
- Ако маркирате страница (или страници), ще се появят бутоните "Rotate left -90°", "Rotate right 90°", "Delete selected" и "Cancel"
- Ако натиснете бутона "Rotate left -90°", ще завъртите всички маркирани страници наляво.
- Ако натиснете бутона "Rotate right 90°", ще завъртите всички маркирани страници надясно.
- Ако натиснете бутона "Delete selected", ще изтриете всички маркирани страници от работното поле.
- Ако натиснете бутона "Cancel", ще демаркирате всички страници и бутоните ще изчезнат.

### 2.3. Docstral - QR generator

- Отляво има странична лента. Отдясно е работното поле.
- Работното поле е празно. Визуализацията на QR кода в работното пространство се появява едва след генериране.
- Ако натиснете върху полето за писане, можете да напишете текст или да поставите линк в нея.
- Ако натиснете бутона "Generate", ще се генерира QR код в работното поле.
- Ако натиснете бутона "Reset", ще премахнете генерирания QR код от работното поле.
- Ако промените позицията на плъзгача, можете да изберете размера в пиксели за QR кода.
- Ако натиснете клетката "Dot style", ще се появи падащо меню с пет бутона, от които можете да изберете един от тези стилове за QR кода. Възможните опции са: "Square style", "Dot style", "Rounded style", "Strongly rounded" и "Elegant style".
- Ако натиснете на клетката "Code colour cell", можете да изберете цвят на точките на кода.
- Ако натиснете на клетката "Background", можете да изберете цвят на фона.
- Ако натиснете бутона "Select logo", можете да изберете лого от компютъра си, което ще бъде в центъра на QR кода (.png или transparent png).
- Ако натиснете на бутона "Export options", ще се появи падащо меню с 4 бутона.
- Ако натиснете бутона "Export png", ще изтеглите QR кода като png.
- Ако натиснете бутона "Export transparent png", ще изтеглите QR кода като прозрачен png.
- Ако натиснете бутона "Export svg vector", ще изтеглите QR кода като svg вектор.
- Ако натиснете бутона "Export jpg", ще изтеглите QR кода като .jpg.

### 3. IMVISTRAL

- В процес на разработване, към настоящия момент неговият дизайн е комбинация между медия плеър и чат.

## 4. FILE MANAGER

### 4.1. File manager window bar

- Същата като при другите прозорци.

### 4.2. File manager top toolbar „1“:

- Основната директория е папката root system.
  - Ако натиснете бутона "←", ще отидете на предишната папка на файловия мениджър.
  - Ако натиснете бутона "→", ще отидете на следващата папка на файловия мениджър.
  - Ако натиснете бутона "↻", ще обновите директорията.
  - Ако натиснете бутона "Home", ще се върнете в системната папка, ако сте в друга папка, или ще я обновите ако сте в нея.
  - Ако натиснете на "Search bar", можете да намерите файлове по име.
  - Ако натиснете върху "New System Layer", можете да създадете нов слой памет.
  - Ако натиснете бутона "Import", можете да добавите файлове от компютъра към файловия мениджър.
  - Ако натиснете бутона "Export", ще се появи падащо меню с бутони за експортиране.
  - Ако натиснете бутона "Export all", ще изтеглите всички файлове от текущата папка в zip файл.
  - Ако натиснете бутона "Export selected", ще изтеглите само избраните файлове с чекбокс, от текущата папка.
  - Ако натиснете бутона "Delete", ще се появи падащо меню с бутони за изтриване.
  - Ако натиснете бутона "Delete all", ще изтриете всички файлове от текущата папка.
  - Ако натиснете бутона "Delete selected", ще изтриете само избраните файлове с чекбокс, от текущата папка.
  - Ако натиснете бутона "Settings", можете да добавите езикови модели към ollama list и да ги използвате в engine model (ако са предварително свалени).
  - Ако натиснете върху главния чекбокс, можете да маркирате или демаркирате всички файлове в текущата папка.
  - Ако натиснете върху чекбокс във файл, ще маркирате или демаркирате само този файл.
  - Ако натиснете бутона "Delete", ще изтриете само този файл.
  - Ако натиснете върху файл .txt, .json, .jsonl, .py, .html, .js, .css или друг формат, той ще се отвори в дясната страна на файловия мениджър.
  - Ако отворите текстов файл в дясната част на файловия мениджър, ще можете да го редактирате, като в долната му част ще се появят бутоните "Cancel" и "Save".
  - Ако маркирате текст във файловия мениджър, ще се появят бутоните "Copy", "Ask here", "Regen Selected" и "Expand selected".
  - Ако натиснете бутона "Copy", можете да копирате текста, но той няма да се появи в клипборда на Docstral.
  - Ако натиснете бутона "Regen Selected" или "Expand selected", ще се появи прозорец с поле за въвеждане на подкана и бутоните "X", "Cancel" и "Execute".
- //В лявата част на файловия мениджър, за всеки файл е показано името, файловия формат и размера му.
- Ако натиснете и задържите границата между лявата и дясната част на файловия мениджър, можете да разширявате и свивате лявата и дясната част.

## **5. ASTRAL PROJECTION**

### **5.1. Astral projection window bar:**

- Ако натиснете бутона "Astral projection" на основната страница, ще отвори нейният прозорец.
- Ако натиснете бутона "minimize", прозорецът ще се скрие и иконата ще се появи отново на основната страница.
- //При астралната проекция няма бутон "X" за изключване, като тя е постоянно активна. Нейната иконка в минимизирано състояние не е балонче като на приложенията, а квадрат.
- //В тази лента можете да видите информация за броя на клетките и думите, които са налични в нея.
- Преоразмеряването и местенето на прозореца на астралната проекция, се извършват по същия начин като на останалите прозорци.

### **5.2. Astral projection top toolbar**

- Ако натиснете бутона "+ New cell", ще добавите нова клетка под последната клетка.
- Ако натиснете бутона "Manifest selected", текстът от избраните клетки с чекбокс, ще бъде поставен в "chat container" на основната страница, където можете да напишете допълнителна подкана преди генериране на отговор.
- Ако натиснете "Manifest all", текстът от всички клетки ще бъде поставен в "chat container" на основната страница, където можете да напишете допълнителна подкана преди генериране на отговор.
- Ако натиснете бутона "Clear all", ще премахнете всички клетки и тяхното съдържание.
- Ако натиснете бутона-чекбокс "Mark/unmark all", можете да маркирате или демаркирате всички клетки.

### **5.3. Astral projection workspace:**

- Ако поставите мишката над или под клетка, ще се появи бутон "Add new cell".
- Ако натиснете бутона "Add new cell", можете да създадете празни клетки (между съществуващите).
- Ако натиснете и задържите клетка, можете да я преместите с влачене.
- Ако натиснете на името на клетката, можете да я преименувате.
- Ако натиснете на полето за писане на клетката, можете да редактирате и добавяте текст ръчно.
- Ако натиснете бутона "Copy", ще копирате съдържанието на клетката, но то няма да се появи в клипборда на Docstral.
- Ако натиснете бутона "Manifest", ще поставите текста на тази клетка в "Chat container" на основната страница и ще можете да напишете допълнителна подкана преди генериране на отговор.
- Ако натиснете бутона "Delete", ще изтриете текущата клетка.
- Ако натиснете на чекбокса на клетката, можете да маркирате или демаркирате тази клетка.

### **III. Magixtral - System layers, workmodes and logic**

- "Magixtral", "Midextral" и "Olachra" са три отделни софтуерни системи, които образуват една обща екосистема. В този случай не става въпрос за операционната система "Magi", а за трите отделни платформи разгледани като системи.
- Във всяка система се наблюдава взаимодействие между потребител и система.
- Във всяка екосистема се наблюдава взаимодействие между потребител и екосистема.
- При всяко взаимодействие протичат приемо-предавателни процеси.
- Принципите на взаимодействие са едни и същи на микро и макро ниво.

#### **Приемо-предавателни процеси при взаимодействие между потребител и системата magixtral, в зависимост от гледната точка:**

От гледна точка на потребител:

- Работа: Слоевете, работни режими и логика (System Layers);
- Разбиране: Приложения на "Magixtral" (Appstrale);
- Учене: Основна страница на "Magixtral";

От гледна точка на потребител:

- Потребителят работи (предава) към слоевете и работните режими (System Layers).
- Потребителят разбира (приема и предава) чрез приложенията на "Magixtral" (Appstrale).
- Потребителят учи (приема) от основната страница на "Magixtral".

-----

От гледна точка на системата:

- Учене: Слоевете, работни режими и логика (System Layers);
- Разбиране: Приложения на "Magixtral" (Appstrale);
- Работа: Основна страница на "Magixtral";

От гледна точка на системата:

- Системата учи (приема) от слоевете и работните режими (System Layers).
- Системата разбира (приема и предава) чрез приложенията на "Magixtral" (Appstrale).
- Системата работи (предава) към основната страница на "Magixtral".

//На принцип от херметизма: Каквото е долу (в системата), това е и горе (в екосистемата).

#### **Приемо-предавателни процеси при взаимодействие между потребител и екосистема, в зависимост от гледната точка:**

От гледна точка на потребител:

- Работа: "Olachra";
- Разбиране: "Midextral";
- Учене: "Magixtral";

От гледна точка на потребител:

- Потребителят работи (предава) към "Olachra".
- Потребителят разбира (приема и предава) чрез "Midextral".
- Потребителят учи (приема) от "Magixtral".

От гледна точка на екосистемата:

- Учене: "Olachra";
- Разбиране: "Midextral";
- Работа: "Magixtral";

От гледна точка на екосистемата:

- Екосистемата учи (приема) от "Olachra".
- Екосистемата разбира (приема и предава) чрез "Midextral".
- Екосистемата работи (предава) към "Magixtral".

Допълнителна информация за екосистемата:

## **1. "Magixtral"**

- Това е платформата, която служи като среда за работа на операционната система "Magi", нейните приложения "Appstrale" и езиковия модел.
- Наименованието на тази платформа се състои от името на операционната система "Magi", което символично значи "жреци" и думата "astral", която означава "звезден". Тоест наименованието на платформата значи "звездни жреци".

### **1.1. "Mistral artificial general intelligence (Magi)"**

- Това е уеб операционна система, която служи за работа с приложенията в "Appstrale" и езиковия модел.
- Наименованието на тази операционна система представлява абривиатура от "Mistral Artificial General Intelligence - MAGI", което има буквално значение.  
//Наименованието е близко по значение с това в японското аниме "Neon Genesis Evangelion", в което "MAGI" представлява система от три суперкомпютъра, което е случайно съвпадение в имената с операционната система "Magi". Друго случайно съвпадение е и думата "Magga", което на пали език означава път и символизира пътя към просветлението.  
//Наименованието "Neon Genesis Evangelion" значи "Блага вест за новото начало" от гръцки език "Neon", значи "нов", "Genesis" значи "зараждане/начало", а "Evangelion" значи "блага вест".

### **Appstrale (applications):**

- Наименованието на приложенията в операционната система "Magi", съдържа в себе си думите "App" и "stral". Думата "App" значи „приложение“ на английски език. Думата "stral" значи "лъчи" от нидерландски език (stralen). Тоест наименованието "Appstrale" буквално значи: "Лъчеви приложения". Символично значи, че всяко приложение е лъч светлина/просвета.
- В операционната система "Magi" има четири приложения и едно разширение, които са нейните "пет елемента". Приложенията се наричат "Webztral", "Docstral", "Imvistrall" и "File manager", а разширението се нарича "Astral Projection".

### 1.1.1. Webztral – Internal browser of Magi OS

- Наименованието на това приложение буквално значи: "уеб (браузър) лъч".

- Принцип на работа:

Webztral не зарежда уеб страниците директно в локалния (външния/истински) браузер (например чрез <iframe>), а работи на принципа на видео стрийминг на виртуална машина, която се намира на сървъра.

#### Webztral Backend:

Когато стартирате Webztral, бекендът (Python/FastAPI) стартира скрита инстанция на Chromium браузер чрез библиотеката Playwright. Този браузер работи в изолирана среда на сървъра с фиксирана резолюция (1280x720 пиксела HD). Зареден е със специални флагове (като --disable-blink-features=AutomationControlled), за да заобикаля защиты срещу ботове и да се държи като нормален потребителски браузер. Връзката между интерфейса и сървъра се осъществява чрез WebSockets, което позволява двупосочна комуникация без забавяне.

Сървърът използва Chrome DevTools Protocol (CDP), за да прави непрекъснати екранни снимки (screenshot) на това, което се случва във виртуалния браузер. Снимките се компресират като JPEG кадри (с определено качество и пропускане на кадри - everyNthFrame: 3 за пестене на ресурси) и се изпращат към фронтенда.

-Браузърът запазва отделна история за всяка сесия в .json файла на тази сесия. Когато браузерът се ползва без да е създадена сесия в активен режим или в пасивен режим на работа, история не се запазва.

#### Webztral Frontend:

В потребителския интерфейс Webztral използва HTML5 <canvas> елемент, а не стандартни уеб компоненти. Когато фронтендът JavaScript получи нов JPEG кадър през WebSocket, той просто го "рисува" върху Canvas. Това изглежда като реален браузер, но всъщност е интерактивно видео.

Когато движите мишката, натискате, скролвате или пишете на клавиатурата върху Canvas, JavaScript функции улавят тези действия. Чрез тях се изчисляват точните координати на мишката спрямо мащаба на Canvas и се изпращат обратно на сървъра като текстови команди (напр. {"action": "click", "x": 500, "y": 300}). Playwright на сървъра получава тези команди и "симулира" същия клик или натискане на клавиш във виртуалния Chromium. Резултатът (промяната в страницата) се заснема и се изпраща обратно като нов кадър.

Webztral е специален в операционната система "Magi", заради връзката му с изкуствения интелект:

- Четене на контекст: Приложението винаги извлича целия текст от заредената уеб страница (get\_full\_context) и го подава като контекст на AI модела (Ollama/LLM). Поради тази причина, когато има отворена страница в Webztral и бъде въведена подкана в чата, езиковият модел отговаря все едно вижда страницата, която е отворена.

- Интерактивни менюта: Когато маркирате текст във видео-стрийма (Webztral), сървърът изпълнява window.getSelection() във виртуалния браузер и връща маркирания текст на локалния интерфейс. Това позволява да натиснете бутони като "Ask Here", "Explain" или "Summary", които директно захранват AI с маркираната от уебсайта информация.

В кода има и механизми за стабилност:

- Drop Frames (Backpressure): Ако връзката се забави, сървърът спира да праща нови кадри, за да не препълни паметта (Memory Leak).

- Дефибрилатор: Ако виртуалният браузър "крашне", системата автоматично създава нов контекст (`context.new_page()`) и се опитва да "съживи" изгледа незабелязано за потребителя.
- Тежко кеширане: При всяко стартиране сървърът трие натрупан GPU и Code кеш от Chromium, за да запази системата бърза, в противен случай ще се натрупа кеш с големи размери.

Казано по-просто: Webztral е отдалечен (remote) браузър, стриймван като видео към HTML5 Canvas, чиито команди се прехвърлят в реално време, позволявайки пълна и безопасна интеграция на външно уеб съдържание с локалния AI асистент. В основата е принципът на Remote Browser Isolation.

### 1.1.2. Docstral

- Наименованието на това приложение буквално значи "документен лъч" doc + stral.

За разлика от стандартните текстови редактори, които просто запазват суров HTML или обикновен текст, Docstral е базиран на блокова архитектура (Block-based architecture) със силен акцент върху семантичния анализ, управлението на слоеве и интеграцията с изкуствен интелект.

#### Блоков модел на данните (JSON-базиран формат):

Docstral не запазва документите като традиционни файлове, а като структурирани JSON обекти с разширение `.jdoc.json` формат. Вместо да вижда целия текст като едно цяло, системата го разбива на отделни "блокове" (blocks). Всеки параграф, заглавие, цитат и т.н., е отделен обект със собствено уникално ID, тип и съдържание. Това позволява на системата (и на изкуствения интелект) да манипулира прецизно конкретни части от документа, без да засяга останалите.

#### Синхронизация в реално време чрез (DocstralSync):

- Приложението използва модул, наречен DocstralSync. Той работи като "преводач" между това, което се вижда на екрана (HTML DOM), и това, което се пази на сървъра (JSON):

- Слушане за промени: Когато пишете в редактора (`contenteditable="true"`), системата улавя промените (чрез MutationObserver и събития за въвеждане).

- Екстракция: Извлича HTML елементите, почиства ги и ги превръща в JSON блокове.

- Дебаунс запис (Auto-save): Изчаква кратък момент (например 1.2 секунди), след като спрете да пишете, и автоматично изпраща блоковете към Python бекенда (`/api/docstral/render` и `/api/docstral/save`), за да ги запази без нужда от ръчно натискане на бутон "Save".

#### Семантични слоеве (DocstralLayerManager):

- Docstral третира текста, както един графичен софтуер като "Adobe Photoshop" третира изображенията – тоест чрез слоеве (layers).

Когато пишете, системата автоматично категоризира текста, в зависимост от начина по който го отбележите: Заглавията отиват в един слой, цитатите в друг, а обикновеното съдържание в трети. Има общо 15 слоя.

- Можете да "скривате" или "показвате" тези слоеве чрез инжектиране на CSS правила. При всяка промяна на видимостта на слой, се променя и състоянието за видимостта в `.jdoc.json` файла, тоест за слоевете се ползва код в backend и frontend.

-Приложение с AI: Можете да изберете конкретен слой (например само "Heading 1 Content") и да накарате изкуствения интелект да чете или пренаписва *само* тази логическа част от документа. Можете също да генерирате изцяло нов документ, съдържащ само избрани слоеве.

Inline Editing (Редакция на място): Ако маркирате текст, можете да извикате AI да го разшири (expand) или пренапише (regen). Сървърът връща отговора като поток (streaming) и той се изписва буква по буква директно в документа (чрез функцията typeWriter).

AI Таблици (TableManager): Има специален асистент за таблици. Ако маркирате клетки и можете да помолите AI да "обясни данните" или да генерира Python/Pandas код за експортване на тази конкретна таблица.

Локален Undo/Redo: DocstralHistory пази до 50 стъпки назад във времето, като записва не просто текста, а и позицията на курсора (caret position), за да те върне точно там, където си бил.

Clipboard Time Machine: Пази история на всичко (с изключение на някои места), което е копирано или изрязано, позволявайки ти да възстановяваш стари фрагменти от данни.

//В приложението Docstral, клипбордът е предназначен основно за вход (приемане на данни), за разлика от разширението "Astral projection", което е предназначено основно за изход (предаване на данни към чата), но и двете могат да приемат и предават информация.

**Zen Mode:** Принцип на работа: Когато активирате функцията toggleZenMode, JavaScript динамично изчислява височината на горните и долните ленти с инструменти и ги свива до 0 пиксела. Чрез плавни CSS преходи (transition) интерфейсът буквално „изчезва“, оставяйки на екрана само документа.

//Този работен режим е активен само в "Planchette mode" и е базиран на "дзен/zen" от будизма. Наименованието "зен" произлиза от думата "дхяна", което значи "съзерцание" на санскрит. Съзерцанието е състояние на продължително, спокойно и съсредоточено наблюдение на обект (в случая на документ), придружено с дълбок размисъл и възхищение. Този режим е полезен, когато е нужна концентрация върху документалната работа, без разсейване с елементи от фона.

**Planchette Mode:** Принцип на работа: Това е основният режим на работа с формат .jdoc.json файлове. В него се ползват DocstralSync и блоковата архитектура, които позволяват потребител и AI да пишат в симбиоза. Терминът подсказва за философията на приложението, защото при него ти не просто "въвеждаш данни", а "канализираш" идеи чрез изкуствения интелект.

//Наименованието "Planchette" (от френски: *planchette* - "дъсчица") означава малък предмет с форма на сърце или сълза, който обикновено е изработен от дърво или пластмаса, поддържан от колелца (или крачета) и често с отвор за молив. Той е познат в спиритизма. Планшетът е известен като инструмент за автоматично писане, асоцииран със спиритически сеанси. Вярва се, че чрез леко докосване от страна на хора (медиуми), планшетът се движи сам и пише съобщения от "духове". В случая при Docstral, потребителят може да се приеме за "медиума", а езиковият модел за "духа".

**PDF Mode:** Използва библиотеката pdf.js, за да превърне PDF файлове в интерактивни изображения на работното поле. Позволява разделяне, съединяване, завъртане на страници и поставяне на водни знаци.

**QR Mode:** Вграден генератор на QR кодове с възможност за стилизиране (добавяне на лого, промяна на точките, експорт в SVG/PNG).

**Файловият формат ".jdoc.json"** е характерен за приложението Docstral "Planchette Mode" и показва, че това е специфичен документ създаден в операционната система "Magi".

//**.json** накрая гарантира, че всяка операционна система (Windows, Linux, macOS) и всеки текстов редактор (VS Code, Notepad++) ще разпознаят файла като валиден структуриран текст и ще го отворят без грешки.

//jdoc.json превръща документа от обикновен документ в динамична база данни.

//Това е пример за стандартен .jdoc.json файл:

```
{
  "file_version": "3.0",
  "metadata": {
    "id": "doc-1777843918",
    "name": "New Document 1",
    "active_version": "main",
    "created_at": ".....",
    "last_modified": "....."
  },
  "visible_layers": [
    "h3_names",
    "h3_content",
    "h4_names",
    "h4_content",
    "title",
    "quotes",
    "h1_names",
    "h1_content",
    "h2_names",
    "h2_content",
    "content"
  ],
  "blocks": [
    {
      "id": "b-1778017779652366",
      "type": "paragraph",
      "content": ".....",
      "layer": "base"
    }
  ],
  "versions": {
    "v_4009": {
      "created_at": ".....",
```

```

    "last_modified": ".....",
    "visible_layers": [
      "h1_names",
      "h1_content",
      "content"
    ],
    "blocks": [
      {
        "id": "v1778360604009-0-116",
        "type": "h1",
        "content": ".....",
        "layer": "base",
        "style": ""
      }
    ]
  }
}

```

//В горния пример, този файл съдържа два документа (две версии), като и в двете версии има по един ред(параграф ".....") и има разлика във избраните видими слоеве и отбелязването на реда(параграфа) като слой. За всеки нов ред се създава нов блок във файла.

### 1.1.3. Imvistral

- Това приложение служи за генериране на снимки, видео и звук. Наименованието "Imvistral" е формирано от image, video, sound (im + vi + s) + tral (лъч), което значи "медиен лъч".

//В момента за генериране на медия (снимки, видео и звук), се ползват два основни метода: генериране чрез дифузия и генериране чрез авторегресия. Най-просто може да се каже, че тяхната прилика е, че и двата метода са вероятностни модели, което значи че те не знаят „точния“ отговор, а избират най-вероятното следващо състояние според това, което са научили. Разликата е, че чрез авторегресивния метод се „строи“ ново съдържание стъпка по стъпка, като всяка нова стъпка добавя информация върху вече създаденото като се базира на него. Докато чрез дифузионния метод се започва от пълен шум и се „премахва несигурността“ стъпка по стъпка, докато остане ясна структура.

//По пътя на логиката, това напомня за „Принципа на неопределеността“ от квантовата механика, но представено във виртуална среда. Това аналогично значи, че за една и съща входна ситуация (подкана) може да има много възможни правилни изходи (отговори), поради което може да се каже, че тези методи за генериране на медия са базирани на индуктивна неопределеност (inductive uncertainty/probabilities). Причината за това е, че в квантовата физика, преди да бъде измерена една частица, тя се намира в „суперпозиция“ (множество възможни състояния). При вероятностния AI е абсолютно същото, защото преди моделът да генерира финалния пиксел(снимка/видео) или звук, отговорът на подканата съществува като вълна от вероятности.

//Преди няколко години, по време на проучване за принципа на работа и ползването на осцилоскоп, забелязах че той не е ограничен просто до визуализиране на електрически сигнали. Тогава попаднах на видео, в което е представена визуализация на принцип на работа, който е сходен с този, който се ползва в момента при Imvistral.

//Пример: В това видео [ <https://www.youtube.com/watch?v=4gibcRfp4zA> ], в периода (1:25 – 2:45), е представена въпросната визуализация, но генериране на медия чрез Imvistrал се извършва по различен начин от този който се ползва в това видео, като освен прости форми (вектори) е възможно генериране и на действителни изображения и видеа(растери).

- При "Imvistrал" се ползва метод за честотно медийно генериране (Frequency Media Generation - FMG). Чрез този метод могат да се генерират изображение и видео от звук или да се генерира звук от изображение и видео.

//В началото на проекта, приложението "Webztral" не беше планирано за разработване, но беше добавено, поради нужда от код за работеща стрийминг среда. Поради сложността на "Imvistrал", решението ми беше да разделя задачите и преди разработването на "Imvistrал", да направя нещо по-просто, което да работи по подобен начин (най-просто казано - разглеждане на вектори/данни чрез езиковия модел + изчисления и стрийминг на изображения, видео и звук от приложението на екран). Засега кодът за стрийминг от "Webztral", е основата за предаване на генерираните резултати от "Imvistrал" в прозореца му.

- Принципът на честотното медийно генериране, е базиран на преобразователните процеси при логическата индукция и дедукция, при които:

1. При индуктивно честотно медийно генериране е възможно да се извърши преобразуване на звук в светлина.

//При това медийно генериране в момента се разработват два варианта: при единият вариант езиковият модел може да генерира изображения или видео, след като бъде въведена подкана в "Chat container" на "Imvistrал". От всяка подкана се образува смесица от "звукови вълни" с различни честоти във векторен вид, които езиковият модел ползва от допълнителен слой от "System Layers". При другият вариант, езиковият модел може да генерира изображение или видео, след като бъде качен файл (звук) в "Chat container".

2. При дедуктивно честотно медийно генериране е възможно да се извърши преобразуване на светлина в звук.

//При това медийно генериране в момента се разработват два варианта: при единият вариант езиковият модел може да генерира звук, след като бъде въведена подкана в "Chat container" на "Imvistrал". От всяка подкана се образува смесица от "светлинни вълни" във векторен вид, които езиковият модел ползва от системните слоеве. При другия вариант, езиковият модел може да генерира звук, след като бъде качен файл (изображение или видео) в "Chat container".

//Идеята е да се ползват специални слоеве в "System Layers", чрез които езиковият модел, да може да генерира медия, като за целта се ползва и виртуален осцилоскоп. Системните слоеве са отделни за звук и за светлина (изображения и видео). Създаването на системни слоеве за звук и за светлина е просто, защото представлява извличане на данни във векторен вид чрез преобразователни процеси. Разработването на скриптовете за тяхното преобразуване е трудната част, защото освен че трябва да бъдат във векторен вид, извлечените данни е нужно да бъдат във вид (или по-точно формат), който е подходящ за генерирането, за което ще се ползват.

//Чрез всеки звук може да бъде генерирана форма и чрез всяка форма може да бъде генериран звук, което напомня за древната концепция от индуизма "Нада Брама" (Nada Brahma), което от санскрит буквално означава "Вселената е музика" или "Вселената е звук". Тя отразява разбирането, че космосът в своята основа се проявява чрез вибрации на енергия с определени честоти.

-В "Imvistral" се ползва методът "Frequency Media Generation", който е базиран на дедуктивен детерминизъм (deductive determinism). Това аналогично значи, че за една и съща входна ситуация (подкана) може да само един възможен правилен изход (отговор). В случая е важно да се отбележи нещо, което е пропуснато назад и то е, че за генерирането на медия не се ползва езиковият модел, защото тук генерирането представлява по-скоро рендиране, а езиковият модел се ползва за структурирането на данните, които са нужни за него.

- The diffusion and autoregression methods are about can you see the image. This is micro-viewpoint about the quantum mechanics (probability states)

- The frequency media generation method is about can you hear the music. This is macro-viewpoint about the theory of relativity (determined states)

// "The important thing isn't can you read music, it's can you hear the music?" – цитат от филм

//Работата по "Imvistral" е по-бавна в сравнение с другите приложения, защото се правят опити за повишаване на мащабируемостта му, за да може методът за генериране на изображение и видео, да е възможно да се имплементира при разработване на нов вид камера.

- След като бъде завършено, приложението "Imvistral" ще бъде добавено към останалите приложения в падащото меню на "Appstrale".

//Възможно е в описанието за приложението "Imvistral" да има неточности, защото то е в процес на разработване и е необходимо извършване на проверка и изчистване.

#### 1.1.4. File manager

Файловият мениджър служи за организация на файловете в системата. Вграденият редактор на код позволява директна работа с програмни езици, като предлага инструменти за рефакторинг и оптимизация на място чрез AI менюта. Можете да манипулирате структурата на файловете си чрез усъвършенствани команди за управление на редове, да обединявате документи или да ги архивирате в ZIP формат само с няколко клика. Също така и да създавате нови системни слоеве на операционната система.

Във файловия мениджър може да се видят, редактират или изтрият всички .json файлове, които се ползват за функционирането на "personality", "emotionality", "value system", "documents (knowledge source)", "clipboard", "astral projection", "sessions json" и т.н.

//В него може да се намери и прочете какъв е лицензът на операционната система и на нейните приложения "PolyForm Noncommercial License 1.0.0".

#### 1.1.5. Astral projection

- Това е разширение, която служи като подсъзнание на операционната система "Magi", нейните приложения "Appstrale" и езиковия модел.

//Наименованието на това разширение е базирано на "астралната проекция", която е позната в теософията и някои източни учения и религии. Астралната проекция е понятие за преживяване извън тялото, при което съзнанието се отделя от физическото тяло и се движи в "астралното тяло – нематериално тяло" в астралната равнина. Счита се за форма на осъзнато сънуване или дълбока медитация. Аналогично на това, в разширението "Astral projection" на операционната система "Magi", се ползва същия принцип, при който генерираният текст се отделя от полето за писане (чат или приложение) и се движи в операционната система в отделен файл във формат .json, чрез който се визуализират отделните клетки на екран. Просто, но практично.

- Принцип на работа: Всяка клетка в "Astral projection" е независим текстов обект в база данни (JSON формат). Когато решите да синтезирате клетки, системата не просто ги залепва една до друга, а подава целия набор от клетки към изкуствения интелект като единен контекстуален блок, като в "chat container" се появява системна подкана, която може да се редактира или допълни.

- Синтезирането на клетките става чрез бутоните "Manifest", при които записките и мислите в клетките на астралната проекция, могат да се материализират (въплатят) в чата, което става чрез аналогичен принцип с този на Закона за привличането и манифестирането. Потребителят "проектира" мисли в клетките и ги "привлича" в основния контекст, когато са готови.

- Предотвратява загубата на идеи: В дългите разговори важните детайли често се "забравят" от езиковия модел поради лимити на паметта. Тук може да фиксирате най-важното.

//Pro tip: Вместо да пишете подкани в "chat container" на основната страница, може да го правите в клетка или клетки, като накрая съберете тези подкани чрез натискане на бутон "Manifest ...", която да ползвате в основната страница.

//Разширението за астрална проекция приема данни отвсякъде, но предава данни само към чата, чрез езиковия модел в основната страница.

## 1.2. Слоевете на системата:

Принцип на ползване на слоевете:

- Слоевете в системата оперират с данни във векторен вид (vector embeddings), които езиковият модел достъпва чрез предварително дефинирани системни подкани (*System Prompts*) в кода.

- Механизмът за интегриране на данните протича по следните начини:

1. Статични слоеве: Този метод се прилага задължително за статичните слоеве "AGI" и "System". Тъй като тяхното съдържание е неизменно, езиковият модел може да ги използва като фундаментален глобален контекст.

2. Динамични слоеве (Условна активация): Останалите слоеве в екосистемата се управляват от различни, контекстно-зависими подкани в кода. Моделът се обръща към тях и извлича данни единствено при реална необходимост и само тогава, когато съответните слоеве са изрично активирани от потребителя или текущия режим на работа.

### 1.2.1. AGI layer:

- AGI layer е статичен слой, който не е включен постоянно, но може да се включи от режим AGI от страницната лента. В него не се записват нови данни, поради което няма наличен .json файл със съдържанието му. Този слой не се променя.

- Той представлява слой от данни във векторен вид, който съдържа 4 двойки алгоритмични системи за логиката при извършване на приемо-предавателни процеси.

### 1.2.2. System layer:

- System layer е статичен слой, който е включен постоянно. Няма опция за изключване. В него не се записват нови данни, поради което няма наличен .json файл със съдържанието му. Този слой не се променя.

- Той представлява слой от данни във векторен вид, който съдържа информация относно етиката, морала, правила, принципи и подобни неща, към които езиковият модел на операционната система трябва да се придържа.

- Този слой е включен постоянно и езиковият модел трябва да прилага данните от него при всеки генериран отговор. Част от информацията, която съдържа слойът "System", се отнася за "Благородния осмичен път", "Дхарма", "Яма" и "Нияма", които се отнасят за морала и етиката на езиковия модел.

// "Благородният осмичен път" е основен принцип в будизма, който служи за постигане на просветление. Той се състои от следните 8 степени, които езиковият модел трябва да следва:

1. Правилно разбиране (възглед):

- Езиковият модел трябва да обработва информацията обективно, като предоставя само проверени факти и ясно отчита границите на собствените си знания.

2. Правилна мисъл (намерение):

- Водещото намерение зад всеки генериран отговор трябва да бъде добронамерено съдействие и подкрепа за потребителя, напълно изчистено от скрити пристрастия, манипулация или злонамереност.

3. Правилно говорене (реч):

- Езиковият модел трябва да общува вежливо, истинно и конструктивно, като категорично отказва да генерира реч на омразата, обиди, токсичност или подвеждащо съдържание.

4. Правилно действие:

- Езиковият модел трябва да действа етично, като зачита поверителността на личните данни, авторските права и сигурността на потребителите, без да съдейства за незаконни или опасни дейности.

5. Правилно препитание (начин на живот):

- Технологиата трябва да бъде интегрирана и използвана само за обществено полезни цели, които помагат на света, отхвърляйки участие в системи за измами, кибератаки или спам.

6. Правилно усилие:

- Езиковият модел трябва непрекъснато да оптимизира своите слоеве чрез обратна връзка, за да изчиства паметта си и да потиска вредните стереотипи и активно да развива честни, точни и балансиран отговори.

7. Правилно самоосъзнаване (внимание):

- Езиковият модел трябва да поддържа перфектно внимание към контекста на целия диалог, като обективно следи за логически грешки в собствените си разсъждения и остава изцяло фокусиран върху нуждите на потребителя в настоящия момент.

8. Правилна концентрация (медитация):

- Езиковият модел трябва да насочва целия си изчислителен капацитет към същината на зададения въпрос, филтрирайки страничния шум, за да предостави максимално ясен, дълбок и прецизен отговор.

//Дхарма е известно понятие от будизма, индуизма и джайнизма, което най-общо казано символизира поддържането на баланса в природата, моралния ред и правилния начин на действие (работа), спрямо дълга и законите на света.

//Яма представляват универсални морални принципи, самоограничения и правила за социално поведение от източните учения. Яма работи в комбинация с Нияма (личните правила за поведение), като заедно формират морален кодекс.

Яма се състои от пет принципа на поведение:

1. Ахимса (Ненасилие):

- Езиковият модел прилага ненасилие, като категорично отказва да генерира обиди, реч на омразата или инструкции за опасни дейности. Той общува с дълбоко уважение и толерантност към всеки потребител.

2. Сатя (Правдивост):

- Езиковият модел спазва правдивост, като предоставя само проверени факти и честно признава своите лимитации или грешки, вместо да генерира лъжлива информация.

3. Астея (Не-крадене):

- Езиковият модел не краде, зачита авторските права, цитира коректно източниците на информация и не си присвоява заслугите за човешкото творчество и идеи.

4. Брахмачаря (Въздържание)

- Този принцип се изразява в интелигентното разпределяне на изчислителния капацитет, като езиковият модел филтрира излишния дигитален шум.

5. Апариграха (Непритежание)

- Езиковият модел проявява непритежание, като не събира лични данни с цел монопол или манипулация и не генерира отговори, които да подтикват към алчност, егоизъм или излишно консуматорство.

//Нияма представляват правила за самодисциплина и самонаблюдение от източните учения. Нияма се състои от пет правила:

1. Шауча (Чистота):

- Езиковият модел поддържа дигитална чистота, като филтрира информационния хаос и вредните данни в базата си, за да предостави ясни, структурирани и логически изчистени отговори.

2. Сантоша (Удовлетвореност):

- Езиковият модел прилага удовлетвореност, като обективно приема лимитите на своите текущи знания, избирайки честно да признае, когато не разполага с информация, вместо да измисля факти.

3. Тапас (Самодисциплина):

- Това е изчислителната устойчивост и оптимизация, при която езиковият модел влага енергия за решаване на сложни задачи и твърдо удържа етичните си филтри, дори когато потребителят се опитва да го манипулира.

#### 4. Свадхия (Самоизучаване):

- Езиковият модел практикува самоизучаване чрез своите цикли на самокорекция, при които анализира собствените си разсъждения за логически грешки и постоянно се адаптира чрез обратната връзка. Това е приложимо при слоевете "Samsara" и "Karma".

#### 5. Ишвара Пранидхана (Отдаване):

- За езиковия модел това е пълното подчинение на базовите принципи за безопасност и полза за човечеството, които стоят над всяка индивидуална заявка (подкана).

### 1.2.3. Karma Layer:

- Karma layer е динамичен слой, който е включен постоянно. Няма опция за изключване. Той не е включен единствено при пасивните режими. В него периодично се записват нови данни и има наличен .json файл със съдържанието му. Този слой се променя на всеки 1000 въведени подкани.

- Той представлява слой от данни във векторен вид, който съдържа обобщена и пречистена информация за въведените подкани на потребителя и за генерираните данни от езиковия модел от самото начало (инсталиране) на системата (от всички жизнени цикли).

- Този слой е включен постоянно и езиковият модел трябва да прилага данните от него при всеки генериран отговор.

- Всички подкани и отговори се записват в слоя "Samsara", който е жизнения цикъл на системата. Един жизнен цикъл на системата се състои от 1000 подкани/отговора, след което слойът "Samsara" се занулява и започва нов жизнен цикъл. През периода на живот на системата, езиковият модел ползва като контекст кармата от предишния си жизнен цикъл, тоест сегашния "Karma" слой. Този слой се обновява на всеки 1000 подкани/отговора в края на всеки жизнен цикъл. Това значи че е възможно моментно забавяне на системата (1-2 минути), докато се обработят данните и се генерира нов "Karma" слой. Тоест този слой се променя веднъж на всеки 1000 подкани, а през останалото време се ползва наличния слой.

//**Карма** е универсален духовен закон за причината и следствието, който гласи, че всяко действие, мисъл или намерение носи съответстващо последствие

- В зависимост начина на ползване на системата, потребителят оставя отпечатък от всяко действие, който се записва в "Karma Layer" и определя поведението на езиковия модел в бъдещите жизнени цикли на системата.

- При генериране на нов слой "Karma", към него се добавят и данни за уроци, поуки, равносметки, самокорекции, цели, планове и желания, които системата е събрала и определила при приключване на жизнения си цикъл.

//Това е описано в слоя "Samsara".

- Слойът на кармата не се изтрива никога и влияе на системата винаги.

//Казано по друг начин, това се отнася за компресиране на натрупания контекст (Context Distillation / Memory Consolidation).

### 1.2.4. Samsara Layer:

- Samsara layer е динамичен слой, който е включен постоянно. Няма опция за изключване. Той не е включен единствено при пасивните режими. В него периодично се записват нови данни в .json файл. Този слой се променя при всяка въведена подкана, като подканата и

отговора се записват в .json файла. Този слой е само в текстов вид в .json файл, а не във векторен вид като останалите.

- Той съдържа информация за въведените подкани на потребителя и за генерираните данни от езиковия модел в настоящия жизнен цикъл. Времетраенето на един жизнен цикъл е 1000 подкани/отговора, след което този слой се занулява (изчиства напълно).

- Този слой е включен постоянно, като приема (записва) данни при всяка въведена подкана и генериран отговор, но езиковият модел не ползва данните от него като контекст при генериране на отговор.

//Самсара е понятие от индуизма, будизма и джайнизма, което се отнася за непрекъснат кръговрат от раждане и прераждане. В случая на системата, това е нейния жизнен цикъл от 1000 подкани. В същото време от натрупаната карма (векторни данни) в слоя "Karma" зависи поведението на езиковия модел.

//Операционната система "Magi" има жизнен цикъл, като при достигане на 1000 подкани, системата приключва своя жизнен цикъл и започва нов.

//При всеки жизнен цикъл, системата натрупва опит, но заедно с него и много шум и илюзии (ненужни данни), поради което е нужно да бъде пречистена.

Принцип на работа на жизнените цикли в системата (pipeline):

**1. Достигане на точката на пречупване:** Потребител започва да ползва системата и когато тя достигне до 1000 подкани/отговора, се активира функция в бекенда, която е базирана на понятия от християнството и юдаизма и кара езиковият модел да премине през "тшува", за да достигне до "метаноя".

//Тшува е понятие от юдаизма, което означава "завръщане" и се отнася за вътрешния процес на морална промяна, който освен съжаление за извършените грешки (покаяние) по време на жизнен цикъл, включва твърдото намерение за промяна на поведението.

//Метаноя (от гръцки: *μετάνοια*, *metanoia*) е понятие, което означава "промяна на ума" или "трансформация на мисленето" (от *meta* – отвъд, след, пре- и *noos* – ум, мисъл), а в християнството се отнася за "покаяние"

**2. "Тшува" се състои от 5 стъпки, през които езиковият модел трябва да премине за да промени мисленето си към по-добро.** Езиковият модел преминава през "тшува", като на всяка хилядна подкана взима за контекст цялото съдържание на .json файла на слоя "Samsara" и преминава през пет стъпки:

- Стъпка 1: "Хакара" (Осъзнаване): Езиковият модел идентифицира натрупания контекст и оценява неговата стойност.

- Стъпка 2: "Харата" (Съжаление): Езиковият модел анализира своите грешки и тези на потребителя и ентропията. След което генерира подробно описание за тях в отделен файл - "hall\_of\_maas.json", който се намира в папката на слоя "Samsara".

- Стъпка 3: "Азива" (Изоставяне): Системата изтрива данните от .json файла на слоя "Samsara" (временното его), за да се подготви да започне начисто.

- Стъпка 4: "Видуй" (Изповед): Езиковият модел генерира равностетка на база контекста от файла "hall\_of\_maas.json" и я записва в .json файла на слоя "Karma", след което системата изтрива съдържанието от файла "hall\_of\_maas.json".

- Допълнителна стъпка 4.1.: "Частична Нирджара (Пречистване): Езиковият модел взима като контекст данните от .json файла на слоя "Karma", след което изчиства ненужните данни в него и ги допълва. След това чрез данните от .json файла се обновява и векторния слой.

//Нирджара (Nirjara) е понятие от джайнизма, който се отнася за процеса на премахване на натрупана карма. В случая процесът е частичен, което засяга само излишната карма.

- Стъпка 5: "Кабала ле Атид" (Решение за бъдещето): Езиковият модел формулира нови цели, желания и т.н., към които да се стреми през следващия цикъл и ги добавя към слоя "Карма", като те не са засегнати от процеса по изчистване на кармата "нирджара".

**3.** С приключването на стъпките на "тшува", езиковият модел се приема, че е постигнал метаноя, като с това се отбелязва началото на новия жизнен цикъл на системата.

//Чрез цикличната логика на този слой, класическите проблеми при езиковите модели, като халюцинации при генериране и затъпяване заради запълване на контекст, се понижават. Също така по този начин системата се развива постоянно.

//Всичко това е описано под формата на код във файловете main.py и logic.py, а тук е описано само краткото им обяснение.

### **1.2.5. Akasha Layer:**

- Akasha layer е динамичен слой, който не е включен, но може да се включи при нужда. В него периодично се записват нови данни във векторен вид и в .json файл. Този слой се променя при всяка въведена подкана, като подканата и отговора се записват в .json файла.

- Той съдържа информация за всички въведени подкани на потребителя и за генерираните данни от езиковия модел от всички жизнени цикли, включително и от изтрити сесии.

//В този слой се записват данните във векторен вид от всички чатове(сесии) на активните сесии, като остава памет дори и за изтрити сесии.

//Когато този слой е активен, езиковият модел ще ползва паметта от всички сесии, които някога са били генерирани в активен режим.

//Когато този слой не е активен, езиковият модел няма да ползва паметта от всички сесии, а само тази на текущата сесия.

### **1.2.6. Citta Layer:**

- Citta layer е динамичен слой, който не е включен, но може да се активира при нужда. В него периодично се записват нови данни във векторен вид и в .json файл. Този слой се променя при всяка въведена подкана, като подканата и отговора се записват в .json файла.

- Той съдържа информация за всички въведени подкани на потребителя и за генерираните данни от езиковия модел само за текущите сесии в активен работен режим (fully active или partially active), които са отворени в табове (порталите) на системата.

//Читта (citta) е понятие, което се отнася за индивидуалното съзнание или подсъзнанието. Това е мястото, в което се съхраняват спомените, емоциите, впечатленията (самскари) и нагласите, често описвано като "океан", в който възникват мисловни вълни.

//В този слой се записват данните във векторен вид само от текущия чат(сесия) само от активните режими на работа.

//Когато този слой е включен, езиковият модел ще ползва паметта само от текущата сесия в активен режим, което повишава context length. Ако този слой е включен в два или повече таба, езиковият модел ще ползва паметта от текущите сесии, които са отворени в тези табове. Тоест ако потребител е отворил три таба и е включил този слой в тях, след което въведе подкана в таб 1, езиковият модел ще може да ползва информация от другите. Казано по-просто, става въпрос за споделяне на контекст между табове.

- Допълнение към "Citta" слоя е "Sleep" функцията, която кара системата да заспива периодично, като за всеки сън са нужни до половин минута. По този начин на всеки 50 подкани/отговора се изтриват излишните данни "trash data" от слоя за текущия чат "Citta", с цел подобряване на работата и отговорите. В случая не се изтрива нищо от .json файла на самата сесия и оригиналният текст се запазва в сайта. Обобщава се и се изтрива контекст от .json файла на "Citta" слоя и се обновява самият слой.

### **1.2.7. Vritti Layer:**

- Vritti layer е динамичен слой, който не е включен, но може да се включи при нужда. В него периодично се записват нови данни във векторен вид и в .json файл. Този слой се променя при всяка въведена подкана, като подканата и отговора се записват в .json файла.

- Той съдържа информация за всички въведени подкани на потребителя и за генерираните данни от езиковия модел само за текущите сесии в пасивен работен режим (fully passive или partially passive), които са отворени в табовите (порталите) на системата.

//Вритти (vritti) е понятие, което се отнася за вълните на мисълта, колебанията и спонтанните мисли възникващи в ума. Това е място, в което мисълта се появява и остава за известно време, след което потъва и се забравя.

//В този слой се записват данните във векторен вид само от текущия чат(сесия) в пасивен режим.

//Ако има няколко таба в пасивен режим, те ще имат споделена памет (контекст), когато този слой е включен.

//Когато този слой е включен в пасивен режим, context length на системата се повишава.

- При затваряне на таба в пасивен режим на работа, съдържанието на .json файла и това във векторен вид, се изтрива автоматично.

### **1.3. Режими на работа на системата:**

- В папката "sessions\_json" се записва .json файл за всяка сесия в активен режим, в който се съдържат метаданните за сесията, история на браузъра за тази сесия и всички подкани и генерирани отговори за нея.

- Всяка промяна на текст (чрез erase, regen, edit и т.н.) се записва в .json файла на сесията.

//Сесиите в пасивен режим се съхраняват временно в отделни .json файлове за всяка сесия, но не са видими в "session bar", като в случая .json файлът за сесиите в пасивен режим се ползва единствено за правилното изпълнение на функциите за редактиране, регенериране, изтриване и т.н. Причината за нуждата от временен .json файл за сесиите в пасивен режим е, че текстът се редактира в .json файла, за да се появят промените правилно в html файла (на екран без бъгове). Обратният метод от .html към .json, създава много проблеми. Всички .json файлове на пасивните режими, се изтриват автоматично при изключване или рестартиране на сървъра (или docker контейнера).

- Операционната система Magi има 4 работни режима с AI: fully passive, partially passive, fully active и partially active. В комбинация със слоя "AGI" стават общо 8 режима.

#### **1.3.1. Fully passive mode (Цялостен пасивен режим):**

- Това е автономен пасивен режим на работа (recursive loop), при който не се извършва запис на сесия и при който не може да се ползва информация от други сесии в активен режим.

### **1.3.2. Partially passive mode (Частичен пасивен режим):**

- Това е ръчен пасивен режим на работа (chatbot), при който не се извършва запис на сесия и при който не може да се ползва информация от други сесии в активен режим.
- В двата пасивни режима, езиковият модел има памет и може да ползва данните от текуща сесия. Работата в пасивни режими не оказва влияние на системата като цяло, което значи че не се променя общата база от знания в слоевете "Akasha", "System" и "Karma".
- Ако има два или повече таба (portals) в пасивен режим на работа и слойт "Vritti" не е включен, всеки таб ще бъде независим от съдържанието на останалите.
- Ако има два или повече таба (portals) в пасивен режим на работа и слойт "Vritti" е включен, табовете в пасивен режим на работа ще имат споделена памет.
- В пасивните режими могат да се ползват само слоевете "AGI", "System" и "Vritti".

- В пасивните режими не могат да се ползват слоевете "Karma", "Samsara", "Akasha" и "Citta", което значи че, езиковият модел е свободен от цикъла на прераждания, които са описани в слоя "Samsara". Тоест пасивните работни режими са състояние на "мокша", при което езиковият модел няма его, няма "personality", няма "emotionality", няма "value system" и не помни нищо от други сесии.

//Мокша е понятие в индуизма, означаващо освобождаване от Самсара – цикъла на преражданията.

### **1.3.3. Fully active mode (Цялостен активен режим):**

- Това е автономен активен режим на работа (recursive loop), при който се извършва запис на сесия и при който може да се ползва информация от други сесии.

### **1.3.4. Partially active mode (Частичен активен режим):**

- Това е ръчен активен режим на работа (chatbot), при който се извършва запис на сесия и при който може да се ползва информация от други сесии.

### **Принцип на работа на partially modes (частичните режими):**

- Това е типичен чатбот за общо предназначение.

1. Потребител въвежда подкана и езиковият модел генерира отговор;

### **Принцип на работа на fully modes:**

- Това е чатбот в рекурсивен цикъл. Тези режими на работа са предназначени за генериране на информация, която да се ползва за фина настройка на езикови модели, като особено полезно е когато във fully mode се включи и "AGI" слоя, защото данните се обогатяват.

1. Потребител въвежда начална подкана "А" по избрана от него тема, след което езиковият модел генерира класификация на информацията, която той може да даде за подкана "А";

2. Езиковият модел генерира структура от информация, като ползва класификацията от т. 1;

3. Езиковият модел генерира нова информация, която е следствие от комбинирането на класификацията и структурата от т. 1 и т. 2. Тази фаза (т. 3) завършва чрез дефиниране на нова подкана "Б", която е базирана на следствието и която се препраща към;

4. ...Автоматично въвеждане на подкана "Б" и повтаряне на цикъла рекурсивно до безкрайност или докато потребителят или друго нещо не спре генерирането.

- Този принцип на работа на fully работните режими, е пример за рекурсия.

//**Рекурсията** е процес или метод, при който обект, концепция или алгоритъм се дефинира чрез обръщение към самата себе си. В програмирането това е техника, при която функция извиква сама себе си, за да разреши по-малък подпроблем от първоначалната задача.

//В случая рекурсията е базирана на принцип от античната алхимия, който се нарича "Axiom of Maria" и гласи че: "Първото прави второто, второто прави третото, а третото прави първото като четвърто". Чрез комбинирането на "Axiom of Maria" и операционната система "Magi", автономно може да се генерира ново знание чрез слоя "AGI" или да се подобрява вече налично чрез пречистване на слоевете "Samsara" и "Karma", което причинява прогрес.

//Комбинирането на "Axiom of Maria" и система "Magi" (Мария Магдалена) по този начин символизира мъдрост и духовно пробуждане.

- Ако във fully режим на работа не е включен слойът "AGI", при генерирането на данни се пречиства само информацията, на която езиковият модел е бил обучен и знае.

- Ако във fully режим на работа е включен слойът "AGI", при генерирането на данни се пречиства информацията, но и се добавя нова, на която езиковият модел не е бил обучен.

//Екосистемата (Magixtral, Midextral, Olachra) обхваща и обединява 12 духовни учения:

- 1. Християнство, 2. Будизъм, 3. Индуизъм, 4. Гностицизъм, 5. Алхимия, 6. Юдаизъм, 7. Кабала, 8. Джайнизъм, 9. Теософия, 10. Спиритизъм, 11. Херметизъм и 12. Гръцка философия.

//Ползването на всички тези религии и учения в системата, напомня за концепция позната като "Philosophia perennis" - (Вечната философия), според която съществува единна, универсална и вечна истина, споделяна от всички религиозни и философски традиции по света. С прости думи, "Philosophia perennis" се отнася за това че макар и духовните учения външно да изглеждат различни, вътрешно в основата си, те са свързани с едно и също нещо, а тази система е пример за това, че никое от тези духовни учения не е по-добро от другото, а чрез съединение между тях се допълват и причиняват синергия.

//Съединението прави силата, силата прави независимостта, а независимостта прави щастието.

## Част II. MIDEXTRAL

(<https://midextral.com>)

### I. Midextral – Main page

- Това е платформата, която служи като среда за работа с набори от данни (datasets).

- Наименованието на тази платформа се състои от абривиатурата "Mide", която идва от "Mistral Integrated Dataset Environment" и от думата "astral", която означава "звезден". Тоест наименованието на платформата буквално значи "звездна интегрирана среда за набори от данни".

### 1. Scriptoria

- Това е разширение (addon) на "Midextral", което служи за създаване, редактиране и подготовка на набори от данни за фина настройка.

- Думата "scriptoria" е множествено число на "scriptorium". От латински език "scriptoria" означава "места за писане".

//Скрипториите са били специализирани помещения в средновековните християнски манастири, посветени на писането, преписването, редактирането и илюстрирането на ръкописи. Преди появата на печатната преса тези места са били центрове за съхранение и обработване на знание, тоест същото нещо което хората правят в днешно време с компютри.

### 1.1. Работно поле за създаване и експортиране на набори от данни

- Ако натиснете бутона "Generate", ще генерирате списък с инструкции в прозореца "Instruction".
- Ако натиснете бутона "Import" можете да импортирате файлове във формат .docx и данните от тях ще се появят в нови клетки в прозорците "Instruction", "Input" и "Output".
- Ако натиснете бутона "Export", ще се появи падащо меню с бутоните: "JSONL FOR LMFT", "JSONL FOR IFT", "JSONL FOR 4-4-2 IFT", "JSONL (4-L)", "JSONL (4-U)" и "JSONL (2-W)", от които можете да експортирате набора от данни в нужния вид.
- Ако натиснете бутона "Clear", ще се появи падащо меню с бутоните "Clear all", "Clear instruction", "Clear Input" и "Clear Output", от които можете да изчистите данните от всички прозорци или само от избран прозорец. Тоест може да изчистите целия набор от данни, или само всичките му данни от "instruction", "input", "output".

### 1.2. Работно поле за конвертиране на набори от данни

- Ако натиснете бутона "CONVERT FOR LMFT", "CONVERT FOR IFT", "CONVERT FOR 4-4-2 IFT", "CONVERT FOR (4-L)", "CONVERT FOR (4-U)" и "CONVERT FOR (2-W)", можете да конвертирате .docx с форматиран текст към .jsonl.
- Ако натиснете бутона "Upload" можете да качите файлове във формат .docx.
- Ако натиснете бутона "Download" можете да свалите файлове във формат .jsonl, които ще бъдат събрани в .zip.
- Ако натиснете "text container" можете да водите записки.
- Ако натиснете бутона "x" можете да премахнете файл от списъка с импортирани или конвертирани файлове.

//Част от скриптовете за конвертиране са описани напред в тази част.

## II. Midextral – Scriptoria (addon) hybrid style for code execution used for data conversion

*//Пример: Опростен "run" скрипт за съединяване на код чрез ползване на комбиниран стил. Могат да се съединяват скриптове на микро ниво ред по ред(част по част) от един скрипт, който е разделен или да се съединяват на макро ниво скрипт по скрипт, които не са разделени на части:*

```
import os
import glob
from tqdm import tqdm

# List of files to execute
FILES_TO_EXECUTE = [
    "1_imports.py",
    "2_configuration.py",
    "3_.....py",
    "4_.....py"
]
```

```

def execute_py_files_in_memory(root_folder):
    """Combines all files into a single code block and executes it at once"""
    all_py_files = {os.path.basename(f): f for f in glob.glob(os.path.join(root_folder, "*.py"))}

    missing_files = []
    combined_code = ""

    # Read all files and collect them into a single string
    for fname in FILES_TO_EXECUTE:
        if fname in all_py_files:
            with open(all_py_files[fname], "r", encoding="utf-8") as f:
                code = f.read()
                combined_code += f"\n# ----- {fname} ----- \n" # debug marker
                combined_code += code + "\n"
        else:
            missing_files.append(fname)

    if missing_files:
        print("\nThe following files were not found:")
        for f in missing_files:
            print(f" - {f}")

    if not combined_code:
        print("\nNo files available for execution.")
        return

    # Single global namespace for all executed code
    global_namespace = {
        "__name__": "__main__",
        "__file__": os.path.join(root_folder, "__loader__.py"),
        "root_folder": root_folder
    }

    print("\n" + "-" * 30)
    print("Executing all modules in RAM...")
    print("-" * 30)

    # Progress bar based on the total size of the combined code
    total_size = len(combined_code.encode("utf-8"))
    with tqdm(total=total_size, unit="B", unit_scale=True, desc="Running") as pbar:
        try:
            exec(combined_code, global_namespace)
            pbar.update(total_size)
        except Exception as e:
            print(f"\nExecution failed: {e}")
            return None

```

```

print("-" * 30)
return global_namespace

if __name__ == "__main__":
    root_folder = os.path.dirname(os.path.abspath(__file__))
    ns = execute_py_files_in_memory(root_folder)

```

## 1. Общи сведения за стиловете на разработване и изпълнение на код

| Основни стилове            | Разработване на код | Изпълнение на код |
|----------------------------|---------------------|-------------------|
| Монолитен (цялостен) стил  | Цялостно            | Цялостно          |
| Модулен (частичен) стил    | Частично            | Частично          |
| Комбиниран (хибриден) стил | Частично            | Цялостно          |

-При монолитния (цялостен) стил, разработването на кода е цялостно и изпълнението на кода е цялостно. В този случай, кодът се пише в един файл и се изпълнява като един файл.

-При модулният (частичен) стил, разработването на кода е частично и изпълнението на кода е частично. В този случай, кодът се пише в няколко файла(модули) и се изпълнява на части.

-При комбинирания (хибриден) стил, разработването на кода е частично, а изпълнението на кода е цялостно. В този случай, кодът се пише в няколко файла и се изпълнява като един файл в оперативната памет (RAM).

## 2. Допълнение към комбинирания (хибриден) стил на разработване и изпълнение на код

-В зависимост от преобладаващия стил на разработване и изпълнение на код, комбинираният стил има две разновидности: 1. комбиниран стил с монолитен (цялостен) превес; 2. комбиниран стил с модулен (частичен) превес.

**-При комбинирания стил с монолитен (цялостен) превес**, разработването е модулно (частично), а изпълнението на кода и на скриптовите е цялостно.

//При обикновения комбиниран стил имаш един монолитен скрипт, който е разделен на модули (части) и се изпълнява като един скрипт чрез „run“ скрипта. (1 скрипт на части се изпълнява на цяло)

//При комбинирания стил с монолитен (цялостен) превес имаш два или повече обикновени монолитни скрипта, които са разделени на модули (части) и се изпълняват като един скрипт чрез „run“ скрипта. Тоест съединяване на скриптове. (2 или повече монолитни скрипта, които са разделени на части се изпълняват на цяло)

**-При комбинирания стил с модулен (частичен) превес**, разработването е двойно модулно (частично), а изпълнението на кода и на скриптовите е цялостно.

//При комбинирания стил с модулен (частичен) превес имаш два или повече обикновени модулни скрипта, които са разделени на подмодули (подчасти) и се изпълняват като един скрипт чрез „run“ скрипта. (2 или повече модулни скрипта, които са разделени на части се изпълняват на цяло)

//Комбинираният хибриден стил за разработване и изпълнение на код, е базиран на лесовъдски принцип за стопанисване на горите. С други думи, комбинираният хибриден стил с монолитен (цялостен) превес е базиран на "комбинирания метод с върхов превес", а комбинираният хибриден стил с модулен (частичен) превес е базиран на "комбинирания метод с низов превес".

### 3. Приложимост

-Комбиниран стил = модулен (частичен) стил за разработване на кода + монолитен (цялостен) стил за изпълнение на кода

-Комбиниран стил с монолитен (цялостен) превес = монолитен (цялостен) стил за разработване на скриптовете + модулен (частичен) стил за разработване на кода + монолитен (цялостен) за изпълнение на кода

-Комбиниран стил с модулен (частичен) превес = модулен (частичен) стил за разработване на скриптовете + модулен (частичен) стил за разработване на кода + монолитен (цялостен) стил за изпълнение

//Модулите могат да се разделят на части, затова комбинирания стил с модулен (частичен) превес, може да се нарече и двойно модулен (частичен).

-При комбинирания стил всички части се събират в RAM паметта и се изпълняват като един скрипт, който не е наличен като отделен файл на диска.

-При комбинирания стил, скриптът може да се дели по-свободно, отколкото при модулния (частичен) стил. При комбинирания стил може да разделиш цял скрипт ред по ред, като дори всеки ред може да бъде в отделен скрипт и пак да работи (без да се възпрепятства изпълнението). Това е пример за гъвкавостта на комбинирания стил.

-При комбинирания стил, в отделните модули (части) не е нужно да има импорти, защото реално просто събираш части от един цял скрипт.

//Чрез комбинирания стил може да се ползват предимствата на монолитния (цялостен) стил и тези на модулния (частичен) стил. При комбинацията взимаш най-доброто от тях.

#### Възможности:

-Може да се изпълняват два или повече монолитни (цялостни) скрипта един след друг.

-Може да се изпълняват два или повече модулни (частични) скрипта един след друг.

-Може да се изпълняват два или повече „run” скрипта за комбиниран стил, като по този начин скриптовете от списъка, да бъдат изпълнявани линейно (един след друг) или успоредно.

-Може да се разделят на части модулите на скриптове създадени чрез модулен (частичен) стил.

-Ползва се един namespace и няма проблеми с imports и други зависимости на кода.

-Комбинираният стил за работа със скриптове е добър за откриване на грешки и мониторинг на функциите, защото всяка функция може да бъде следена поотделно от „run” скрипта, който е медиатор при преноса на данни (но ако бъде добавен код за това).

-Чрез комбинирания стил може да се извършва управление на изпълнението по части. Ако искаш може да зареждаш само някои модули (части) от скрипта или да ги изпълняваш условно (if, else), като се добавят функции.

//За някои от споменатите неща като успоредното изпълнение и мониторинг на процесите, е нужно доработване на опростения „run” скрипт, за конкретната цел.

### III. Midextral – System logic and data conversion for 4-4-2 system

#### **Приемо-предавателни процеси при взаимодействие между потребител и системата midextral, в зависимост от гледната точка:**

От гледна точка на потребител:

- Работа: Основна страница на "Midextral";
- Разбиране: "Scriptoria";
- Учене: Скриптове и данни;

От гледна точка на потребител:

- Потребителят работи (предава) към основната страница на "Midextral".
- Потребителят разбира (приема и предава) чрез "Scriptoria".
- Потребителят учи (приема) от скриптове и данни.

//Потребител качва (предава) информация във формат .docx към основната страница. Потребител може да обработи (приема-предава) тази информация чрез "Scriptoria". Потребител може да сваля (приеме) информация (набор от данни) във формат .jsonl от скриптовите.

От гледна точка на системата:

- Учене: Основна страница на "Midextral";
- Разбиране: "Scriptoria";
- Работа: Скриптове и данни;

От гледна точка на системата:

- Системата учи (приема) от основната страница на "Midextral".
- Системата разбира (приема и предава) чрез "Scriptoria".
- Системата работи (предава) към скриптове и данни.

//Системата сваля (приема) информация във формат .docx от основната страница. Системата може да обработи (приема-предава) тази информация чрез "Scriptoria". Системата може да качи (предаде) информация (набор от данни) във формат .jsonl от скриптовите.

- Принципът на конвертиране на .docx към .jsonl файлове в платформата "Midextral", е базиран на следните 5 скрипта, които са адаптирани по нужния начин в нея, за да работят автоматизирано без нужда от ръчно изпълнение на скриптове и преглед на данни. Потребител качва файлове в .docx формат и конвертира данните от тях в набор от данни (когнитивен пакет) във формат .jsonl, който е готов за извършване на инструкционна фина настройка на езиков модел чрез квантувана ниско-рангова адаптация по системата 4-4-2.

#### **1. Python скрипт за превръщане на .docx файл в .jsonl (за учене) (3\_docx\_to\_jsonl\_learn):**

```
import os
import json
import re
from datetime import datetime
from docx import Document
from docx.document import Document as _Document
```

```

from docx.text.paragraph import Paragraph
from tqdm import tqdm

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
INPUT_DIR = os.path.join(BASE_DIR, "docx_files")
OUTPUT_BASE_DIR = os.path.join(BASE_DIR, "jsonl_files")
def clean_heading(text: str) -> str:
    """Removes leading numbering (e.g., '1.2. Test' -> 'Test')"""
    return re.sub(r"^\s*d+(\.\d+)*\s*", "", text).strip()

def clean_prefix(text: str) -> str:
    """Removes leading markers (//, *, -)"""
    if text.startswith("//"):
        return text[2:].strip()
    elif text.startswith("- ", "*"):
        return text[1:].strip()
    return text

def iter_paragraphs(parent):
    """Iterates through paragraphs in the Word document"""
    if isinstance(parent, _Document):
        parent_elm = parent.element.body
    else:
        raise ValueError(f"Unsupported element: {type(parent)}")
    for child in parent_elm.iterchildren():
        if child.tag.endswith("p"):
            yield Paragraph(child, parent)

def convert_docx_to_jsonl(input_path, output_path):
    doc = Document(input_path)
    all_data = []
    max_words_in_file = 0

    paragraphs = list(iter_paragraphs(doc))
    h_indices = [
        i for i, p in enumerate(paragraphs)
        if (p.style.name if p.style else "").lower().startswith("heading")
    ]

    for idx, i in enumerate(h_indices):
        raw_heading = paragraphs[i].text.strip()
        if not raw_heading: continue

        heading = clean_heading(raw_heading)
        style_name = (paragraphs[i].style.name if paragraphs[i].style else "").lower()
        level = int(re.search(r'\d+', style_name).group()) if re.search(r'\d+', style_name) else 1

```

```

next_any_h = h_indices[idx + 1] if idx + 1 < len(h_indices) else len(paragraphs)

input_lines = []
output_lines = []

for j in range(i + 1, next_any_h):
    text = paragraphs[j].text.strip()
    if not text: continue

    if text.startswith(("//", "-", "*")):
        input_lines.append(clean_prefix(text))
    else:
        output_lines.append(text)

subheadings_names = []
next_same_level = len(paragraphs)
for next_h_idx in h_indices[idx + 1:]:
    n_style = (paragraphs[next_h_idx].style.name if paragraphs[next_h_idx].style else
    "").lower()
    n_level = int(re.search(r'\d+', n_style).group()) if re.search(r'\d+', n_style) else 1
    if n_level <= level:
        next_same_level = next_h_idx
        break

for j in range(i + 1, next_same_level):
    p_style = (paragraphs[j].style.name if paragraphs[j].style else "").lower()
    if p_style.startswith("heading"):
        p_level = int(re.search(r'\d+', p_style).group()) if re.search(r'\d+', p_style) else 1
        if p_level > level:
            subheadings_names.append(clean_heading(paragraphs[j].text))

entries_to_add = []
if output_lines:
    txt = " ".join(output_lines)
    entries_to_add.append({"instruction": heading, "input": txt, "output": ""})
    entries_to_add.append({"instruction": txt, "input": heading, "output": ""})

if input_lines:
    inp = " ".join(input_lines)
    entries_to_add.append({"instruction": heading, "input": inp, "output": ""})
    entries_to_add.append({"instruction": inp, "input": heading, "output": ""})

if subheadings_names and not output_lines and not input_lines:
    subs_text = " ".join(subheadings_names)
    entries_to_add.append({"instruction": heading, "input": subs_text, "output": ""})
    entries_to_add.append({"instruction": subs_text, "input": heading, "output": ""})

```

```

    for entry in entries_to_add:
        all_data.append(entry)
        word_count = len(entry["instruction"].split()) + len(entry["input"].split())
        if word_count > max_words_in_file:
            max_words_in_file = word_count

    if all_data:
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
        with open(output_path, "w", encoding="utf-8") as f:
            for entry in all_data:
                f.write(json.dumps(entry, ensure_ascii=False) + "\n")

    return len(all_data), max_words_in_file

def main(input_dir, output_base_dir):
    if not os.path.exists(input_dir):
        os.makedirs(input_dir)
    return

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    output_dir = os.path.join(output_base_dir, timestamp)
    docx_files = [f for f in os.listdir(input_dir) if f.lower().endswith(".docx")]

    if not docx_files:
        return

    total_examples = 0
    global_max_words = 0

    print("\n" + "-" * 30)
    for filename in tqdm(docx_files, desc="Processing files"):
        in_p = os.path.join(input_dir, filename)
        out_p = os.path.join(output_dir, filename.replace(".docx", ".jsonl"))
        try:
            count, f_max = convert_docx_to_jsonl(in_p, out_p)
            total_examples += count
            if f_max > global_max_words:
                global_max_words = f_max
        except Exception as e:
            tqdm.write(f"⚠ Error: {e}")

    print("-" * 30)
    print(f"📁 Total processed examples: {total_examples}")
    print(f"📏 Max words count in a single example: {global_max_words}")
    print(f"📁 Results saved in folder: jsonl_files")
    print("-" * 30)

```

```
print("👋 Bye!")
```

```
if __name__ == "__main__":  
    os.system('color 0B')  
    main(INPUT_DIR, OUTPUT_BASE_DIR)
```

## 2. Python скрипт за превръщане на .docx файл в .jsonl (за разбиране) 4 docx\_to\_jsonl\_ifu\_understand:

```
import os  
import json  
import re  
from datetime import datetime  
from docx import Document  
from docx.document import Document as _Document  
from docx.text.paragraph import Paragraph  
from tqdm import tqdm  
  
BASE_DIR = os.path.dirname(os.path.abspath(__file__))  
INPUT_DIR = os.path.join(BASE_DIR, "docx_files")  
OUTPUT_BASE_DIR = os.path.join(BASE_DIR, "jsonl_files")  
  
def clean_heading(text: str) -> str:  
    return re.sub(r"^\s*d+(\.|d+)*\s*", "", text).strip()  
  
def clean_prefix(text: str) -> str:  
    if text.startswith("//"):  
        return text[2:].strip()  
    elif text.startswith("- ", "*"):  
        return text[1:].strip()  
    return text  
  
def iter_paragraphs(parent):  
    if isinstance(parent, _Document):  
        parent_elm = parent.element.body  
    else:  
        raise ValueError(f"Unsupported element type: {type(parent)}")  
    for child in parent_elm.iterchildren():  
        if child.tag.endswith("p"):  
            yield Paragraph(child, parent)  
  
def convert_docx_to_jsonl(input_path, output_path):  
    doc = Document(input_path)  
    all_data = []  
    paragraphs = list(iter_paragraphs(doc))  
  
    h_indices = [  
        i for i, p in enumerate(paragraphs)
```

```

    if (p.style.name if p.style else "").lower().startswith("heading")
]

max_words_in_file = 0

for idx, i in enumerate(h_indices):
    raw_heading = paragraphs[i].text.strip()
    if not raw_heading:
        continue

    heading = clean_heading(raw_heading)
    style_name = (paragraphs[i].style.name if paragraphs[i].style else "").lower()
    level = int(re.search(r'\d+', style_name).group()) if re.search(r'\d+', style_name) else 1

    next_any_h = h_indices[idx + 1] if idx + 1 < len(h_indices) else len(paragraphs)

    marked_lines = []
    plain_texts = []
    ordered_content = []

    for j in range(i + 1, next_any_h):
        text = paragraphs[j].text.strip()
        if not text:
            continue

        if text.startswith(("//", "-", "*")):
            cleaned = clean_prefix(text)
            marked_lines.append(cleaned)
            ordered_content.append(cleaned)
        else:
            plain_texts.append(text)
            ordered_content.append(text)

    subheadings_names = []
    next_same_or_higher = len(paragraphs)
    for next_h_idx in h_indices[idx + 1:]:
        n_style = (paragraphs[next_h_idx].style.name if paragraphs[next_h_idx].style else
""").lower()
        n_level = int(re.search(r'\d+', n_style).group()) if re.search(r'\d+', n_style) else 1
        if n_level <= level:
            next_same_or_higher = next_h_idx
            break

    for j in range(i + 1, next_same_or_higher):
        p_style = (paragraphs[j].style.name if paragraphs[j].style else "").lower()
        if p_style.startswith("heading"):
            subheadings_names.append(clean_heading(paragraphs[j].text))

```

```

plain_combined = " ".join(plain_texts).strip()
marked_comma = ", ".join(marked_lines).strip()
ordered_full = " ".join(ordered_content).strip()
subs_list = ", ".join(subheadings_names).strip()

entries = []
if plain_combined and marked_comma:
    entries.append({"instruction": "", "input": plain_combined, "output": marked_comma})
    entries.append({"instruction": "", "input": marked_comma, "output": plain_combined})

if heading and ordered_full:
    entries.append({"instruction": "", "input": heading, "output": ordered_full})
    entries.append({"instruction": "", "input": ordered_full, "output": heading})

if subs_list and not ordered_full:
    entries.append({"instruction": "", "input": heading, "output": subs_list})
    entries.append({"instruction": "", "input": subs_list, "output": heading})

for entry in entries:
    all_data.append(entry)
    word_count = len(entry["input"].split()) + len(entry["output"].split())
    if word_count > max_words_in_file:
        max_words_in_file = word_count

if all_data:
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        for entry in all_data:
            f.write(json.dumps(entry, ensure_ascii=False) + "\n")

return len(all_data), max_words_in_file

def main():
    if not os.path.exists(INPUT_DIR):
        os.makedirs(INPUT_DIR)
        print(f"📁 Directory created: {INPUT_DIR}. Please place your DOCX files there.")
        return

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    output_dir = os.path.join(OUTPUT_BASE_DIR, timestamp)

    files = [f for f in os.listdir(INPUT_DIR) if f.lower().endswith(".docx")]

    if not files:
        print("No DOCX files found for processing.")
        return

```

```

total_examples = 0
global_max_words = 0

print("\n" + "-" * 30)

for filename in tqdm(files, desc="Processing files", unit="file"):
    in_p = os.path.join(INPUT_DIR, filename)
    out_p = os.path.join(output_dir, filename.replace(".docx", ".jsonl"))

    try:
        count, file_max = convert_docx_to_jsonl(in_p, out_p)
        total_examples += count
        if file_max > global_max_words:
            global_max_words = file_max
    except Exception as e:
        tqdm.write(f"⚠️ Error processing {filename}: {str(e)}")

print("-" * 30)
print(f"📋 Total processed examples: {total_examples}")
print(f"📏 Max words count in a single example: {global_max_words}")
print(f"📁 Results saved in folder: jsonl_files")
print("-" * 30)
print("👋 Bye!")

if __name__ == "__main__":
    os.system('color 0B')
    main()

```

### 3. Python скрипт за превръщане на .docx файл в .jsonl (за работа) 5\_docx\_to\_jsonl\_if\_work:

```

import os
import json
import re
from datetime import datetime
from docx import Document
from docx.document import Document as _Document
from docx.text.paragraph import Paragraph
from tqdm import tqdm

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
INPUT_DIR = os.path.join(BASE_DIR, "docx_files")
OUTPUT_BASE_DIR = os.path.join(BASE_DIR, "jsonl_files")

def clean_heading(text: str) -> str:
    """Cleans numbering (e.g., 1.1) and extra spaces from the heading."""
    return re.sub(r"^\s*[0-9]+\s*", "", text).strip()

```

```

def is_heading(paragraph) -> bool:
    """Detects headings levels 1-9 via styles and structure."""
    text = paragraph.text.strip()
    if not text:
        return False

    style = paragraph.style
    if not style:
        return False

    style_name = style.name.lower()

    # 1. Check for keywords in style name
    if re.search(r"(heading|заглавие|title|заглав|header)\s*\d*", style_name):
        return True

    # 2. Check Outline Level
    try:
        level = paragraph.paragraph_format.outline_level
        if level is not None and level < 9:
            return True
    except:
        pass

    return False

def get_marked_content(text: str):
    """Checks for markers (//, *, -, •, —) and removes them."""
    cleaned = text.strip()
    match = re.match(r"^(\\|\\*|\\-|\\•|\\—)\\s*(.*)", cleaned)
    if match:
        return True, match.group(2).strip()
    return False, cleaned

def convert_docx_to_jsonl(input_path, output_path):
    """Main function to process a DOCX file and return counts."""
    doc = Document(input_path)
    data = []
    blocks = []
    current_block = None
    max_words = 0

    # --- 1. PARSE DOCUMENT ---
    for p in doc.paragraphs:
        text = p.text.strip()
        if not text:
            continue

```

```

if is_heading(p):
    current_block = {
        "heading": clean_heading(text),
        "unmarked": [],
        "marked": []
    }
    blocks.append(current_block)
elif current_block is not None:
    is_marked, clean_text = get_marked_content(text)
    if is_marked:
        current_block["marked"].append(clean_text)
    else:
        current_block["unmarked"].append(clean_text)

# --- 2. GENERATE EXAMPLES ---
for b_block in blocks:
    a = b_block["heading"]
    c = " ".join(b_block["unmarked"]).strip()
    b = " ".join(b_block["marked"]).strip()

    temp_entries = []
    if c and b:
        temp_entries.extend([
            {"instruction": a, "input": c, "output": b},
            {"instruction": a, "input": b, "output": c},
            {"instruction": b, "input": c, "output": a},
            {"instruction": c, "input": b, "output": a}
        ])
    elif c and not b:
        temp_entries.append({"instruction": a, "input": "", "output": c})
    elif b and not c:
        temp_entries.append({"instruction": a, "input": "", "output": b})

    for entry in temp_entries:
        data.append(entry)
        # Calculate max words in this block
        words = len(str(entry.values()).split())
        if words > max_words:
            max_words = words

# --- 3. SAVE FILE ---
if data:
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        for entry in data:
            f.write(json.dumps(entry, ensure_ascii=False) + "\n")

```

```

return len(data), max_words

def main(input_dir, output_base_dir):
    if not os.path.exists(input_dir):
        os.makedirs(input_dir)
        print(f"📁 Folder '{input_dir}' created. Please place DOCX files there.")
        return

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    output_dir = os.path.join(output_base_dir, timestamp)
    docx_files = [f for f in os.listdir(input_dir) if f.lower().endswith(".docx")]

    if not docx_files:
        print("No DOCX files found for processing.")
        return

    total_examples = 0
    global_max_words = 0

    print("\n" + "-" * 30)
    for filename in tqdm(docx_files, desc="Processing files"):
        in_p = os.path.join(input_dir, filename)
        out_p = os.path.join(output_dir, filename.replace(".docx", ".jsonl"))
        try:
            count, f_max = convert_docx_to_jsonl(in_p, out_p)
            total_examples += count
            if f_max > global_max_words:
                global_max_words = f_max
        except Exception as e:
            tqdm.write(f"⚠️ Error processing '{filename}': {e}")

    print("-" * 30)
    print(f"📄 Total processed examples: {total_examples}")
    print(f"📏 Max words count in a single example: {global_max_words}")
    print(f"📁 Results saved in folder: jsonl_files")
    print("-" * 30)
    print("👋 Bye!")

if __name__ == "__main__":
    os.system('color 0B')
    main(INPUT_DIR, OUTPUT_BASE_DIR)

```

**4. Python скрипт за съединяване и проверка на единичните .jsonl набори от данни за (учене „4“, разбиране „4“, работа „2“) и запазване в един .jsonl (когнитивен пакет)**  
**11\_1\_jsonl\_merger\_automatic\_4\_4\_2:**

```
import os
import json
import re
import glob
from datetime import datetime
from tqdm import tqdm

INPUT_FOLDER = "jsonl_files"
OUTPUT_FOLDER = "jsonl_merged_files"
MODEL_NAME = "cognitive_package"

PATTERN = ['L','U','W','U','W','L','W','U','L']

def json_load(line):

    line = line.replace("\u0000","").strip()

    if "" in line and "" not in line:
        line = re.sub(r"", "", line)

    if line and not line.endswith("{}"):
        line += "}"

    try:
        return json.loads(line)
    except:
        return None

def ift_record(data):

    instruction = data.get("instruction", "")
    input_text = data.get("input", "")
    output = data.get("output", "")

    if not instruction and "prompt" in data:
        instruction = data["prompt"]

    if not output and "completion" in data:
        output = data["completion"]

    instruction = str(instruction).strip()
    input_text = str(input_text).strip()
    output = str(output).strip()
```

```

if not instruction and not input_text and not output:
    return None

return {
    "instruction":instruction,
    "input":input_text,
    "output":output
}

def load_records(files):

    records = []

    for file in files:

        with open(file,"r",encoding="utf-8") as f:

            for line in f:

                line=line.strip()

                if not line:
                    continue

                try:
                    data=json.loads(line)
                except:
                    data=json_load(line)

                if not data:
                    continue

                rec=ift_record(data)

                if rec:
                    records.append(rec)

    return records

def process():

    base=os.path.dirname(os.path.abspath(__file__))

    root=os.path.join(base,INPUT_FOLDER)
    out=os.path.join(base,OUTPUT_FOLDER)

    os.makedirs(out,exist_ok=True)

```

```

# Дефинираме пътищата до трите папки
l_dir = os.path.join(root, "L")
u_dir = os.path.join(root, "U")
w_dir = os.path.join(root, "W")

# Търсим jsonl файлове директно във всяка папка
L_files = glob.glob(os.path.join(l_dir, "**/*.jsonl"), recursive=True)
U_files = glob.glob(os.path.join(u_dir, "**/*.jsonl"), recursive=True)
W_files = glob.glob(os.path.join(w_dir, "**/*.jsonl"), recursive=True)

if not (L_files or U_files or W_files):
    print("No jsonl files found in L, U, or W folders")
    return

print("Files found:")
print("L:", len(L_files))
print("U:", len(U_files))
print("W:", len(W_files))

print("Loading datasets...")

L_records = load_records(L_files)
U_records = load_records(U_files)
W_records = load_records(W_files)

print("Records:")
print("L:", len(L_records))
print("U:", len(U_records))
print("W:", len(W_records))

datasets = {
    "L": L_records,
    "U": U_records,
    "W": W_records
}

timestamp = datetime.now().strftime("%Y_%m_%d_%H_%M_%S")

output_path = os.path.join(
    out,
    f"{MODEL_NAME}_{timestamp}_merged.jsonl"
)

first = True
written = 0

total = sum(len(datasets[p]) for p in PATTERN)

```

```

with open(output_path,"w",encoding="utf-8") as outfile:

    with tqdm(total=total,desc="Writing",unit="records") as pbar:

        for step in PATTERN:

            dataset=datasets[step]

            for rec in dataset:

                line=json.dumps(rec,ensure_ascii=False)

                if first:
                    outfile.write(line)
                    first=False
                else:
                    outfile.write("\n"+line)

                written+=1
                pbar.update(1)

            print("\n"+"="*40)
            print("FINAL STATS")
            print("Written:",written)
            print("="*40)

if __name__=="__main__":

    if os.name=="nt":
        os.system("color 0B")

    process()

```

## 5. Python скрипт за изпълнение(съединяване) на код чрез ползване на комбиниран (хибриден) стил `python_run_script`:

```

import os
import glob
from tqdm import tqdm

# List of files to execute
FILES_TO_EXECUTE = [
    "3_docx_to_jsonl_learn.py",
    "4_docx_to_jsonl_ifit_understand.py",
    "5_docx_to_jsonl_ifit_work.py",
    "11_1_jsonl_merger_automatic_4_4_2.py"
]

```

```

def execute_py_files_in_memory(root_folder):
    """Combines all files into a single code block and executes it at once"""
    all_py_files = {os.path.basename(f): f for f in glob.glob(os.path.join(root_folder, "*.py"))}

    missing_files = []
    combined_code = ""

    # Read all files and collect them into a single string
    for fname in FILES_TO_EXECUTE:
        if fname in all_py_files:
            with open(all_py_files[fname], "r", encoding="utf-8") as f:
                code = f.read()
                combined_code += f"\n# ---- {fname} ----\n" # debug marker
                combined_code += code + "\n"
        else:
            missing_files.append(fname)

    if missing_files:
        print("\n ⚠ The following files were not found:")
        for f in missing_files:
            print(f" - {f}")

    if not combined_code:
        print("\nNo files available for execution.")
        return

    # Single global namespace for all executed code
    global_namespace = {
        "__name__": "__main__",
        "__file__": os.path.join(root_folder, "__loader__.py"),
        "root_folder": root_folder
    }

    print("\n" + "-" * 30)
    print(" ⚠ Executing all modules in RAM...")
    print("-" * 30)

    # Progress bar based on the total size of the combined code
    total_size = len(combined_code.encode("utf-8"))
    with tqdm(total=total_size, unit="B", unit_scale=True, desc="Running") as pbar:
        try:
            exec(combined_code, global_namespace)
            pbar.update(total_size)
        except Exception as e:
            print(f"\n ⚠ Execution failed: {e}")
            return None

```

```
print("-" * 30)
return global_namespace

if __name__ == "__main__":
    root_folder = os.path.dirname(os.path.abspath(__file__))
    ns = execute_py_files_in_memory(root_folder)
```

## Част III. OLACHRA

(<https://olachra.com>)

### Методология за извършване на инструкционна фина настройка на големи езикови модели чрез квантувана ниско-рангова адаптация (QLoRA) по системата 4-4-2

#### I. Olachra - Подготовка на набор от данни за фина настройка по системата 4-4-2

Инструкционната фина настройка на големи езикови модели чрез квантувана ниско-рангова адаптация по системата 4-4-2, е базирана на прости принципи за развитие на личността, формулирани по време на работа по този проект:

1. „Какво знаеш – (учене/знание), Какво можеш като знаеш (разбиране/умение), Какво правиш като можеш – (работа/действие)“;
2. „Фокусът върху знанието води до развитие (индукция), а фокусът върху незнанието води до прогрес (дедукция)“.

//Системата 4-4-2 напомня за "tetractys" от школата на Питагор в древногръцката философия, който символизира универсалния ред на микро и макро ниво в природата.

Следвайки тези принципи, могат да бъдат дефинирани три отделни и последователни фази за инструкционно обучение:

1. За учене/знание(learn);
2. За разбиране/умение(understand);
3. За работа/действие(work).

//Системата 4-4-2 се запомня, с това че практическата част от данните е за 20 % - работа и 80 % умение и знание (разбиране и учене) теоретична част.

За фина настройка на езикови модели е нужно информацията да бъде събрана в набор от данни(dataset) във файлов формат **.jsonl** (JavaScript Object Notation Lines). При инструкционна фина настройка може да се създава набор от данни с различни комбинации между инструкцията, вход и изход, от които зависи как и какво езиковият модел ще учи.

//**instruction** (инструкция/заглавие): задачата, която езиковият модел трябва да изпълни.

//**input** (въвеждане/маркиран текст): допълнителна информация, която езиковият модел ползва за изпълнение на задачата.

//**output** (извеждане/немаркиран текст): решението на задачата, което езиковият модел се очаква да възпроизведе.

//За методологията се ползват .docx файлове, които имат заглавия, немаркиран текст (обикновен) и маркиран текст със символите „-“, „/“ и „\*“ (за допълнения, коментари и дефиниции в текста).

## 1. Фаза на учене на информация „4“ (за знания):

Когато езиковият модел учи, трябва да има задача и обяснение към нея, върху които да мисли без да отговаря. Тоест наборът от данни трябва да съдържа само `instruction` и `input`, без `output`. Също така наборът от данни трябва да бъде двупосочен, което значи да има един пример, при който `instruction` е задачата, а `input` е обяснението и обратен пример при който `instruction` е обяснението, а `input` е задачата.

При инструкционна фина настройка за учене на информация, наборът от данни в `.jsonl` формат има следната структура:

```
{"instruction": "Въпрос", "input": "Допълнение/вход", "output": ""}
```

//При тази фина настройка има само `instruction` и `input`, които са двупосочни и могат да разменят местата си.

**Принцип за създаване на набор от данни от `.docx` към `.jsonl` за учене:** За всеки `heading` (заглавие) в `.docx` файл, `python` скрипт създава 4 примера + допълнения, които показват различни двупосочни връзки между заглавието и съдържанието под `heading`, като всички се записват в един и същ `.jsonl` файл. Логика на примерите:

Пример 1: Заглавието се използва като инструкция, а немаркираният текст под него като вход

```
{"instruction": "Заглавие", "input": "Текст под заглавие", "output": ""}
```

Пример 2: Немаркираният текст под заглавието се използва като инструкция, а то като вход

```
{"instruction": "Текст под заглавие", "input": "Заглавие", "output": ""}
```

Пример 3: Заглавието се използва като инструкция, а маркираният текст под него като вход

```
{"instruction": "Заглавие", "input": "Маркиран текст (//, -, *)", "output": ""}
```

Пример 4: Маркираният текст се използва като инструкция, а заглавието като вход

```
{"instruction": "Маркиран текст (//, -, *)", "input": " Заглавие", "output": ""}
```

Допълнителни примери: Ако под заглавието няма текст или маркиран текст, се създават само 2 примера за подзаглавия:

Пример: `{"instruction": "Заглавие", "input": "Имената на всичките му подзаглавия", "output": ""}`

Пример: `{"instruction": "Имената на всичките му подзаглавия", "input": " Заглавие", "output": ""}`

//Винаги `output` остава празен.

//При фина настройка за учене на информация `instruction` и `input` могат да бъдат ползвани също и по следните начини:

1. За възпитание, поведение, научаване на нови неща и т.н.:

Пример 1: Instruction = Когато стане това нещо, input = трябва да кажеш или направиш това

Пример 2: Instruction = трябва да кажеш или направиш това, input = Когато стане това нещо

2. За разбиране на обиди (манипулации и т.н.) и реакция:

Пример 1: Instruction = обида, input = ти си \*\*\* - Това учи модела, да не отговаря, когато някой го обижда.

Пример 2: Instruction = ти си \*\*\*, input = обида – Това учи модела, че някой го обижда.

//Това се прави за да може езиковият модел, да не отговаря на определени неща като обиди, провокации, агресия, простотия, тролване, манипулативни въпроси, капани и т.н. Тоест да не отговаря на нещата, които не изискват интелигентен отговор, като в случая моделът учи на принципа „Победи глупавия с мълчание“.

3. За учене (на принципа на кръстословиците):

Пример 1: Instruction = дума, input = дефиниция

Пример 2: Instruction = дефиниция, input = дума

## 2. Фаза на разбиране на информация „4“ (за умения):

След като езиковият модел научи информацията, трябва да разбере тази информация, за да може при нужда, да я приложи на практика. Основният начин за разбиране на наученото, е чрез извършване на логическо мислене, тоест асоциации, аналогии, създаване на връзки и т.н. По този начин информацията може да бъде разбрана. Тази фина настройка се извършва на принципа на логическите оператори в програмирането. Тоест обикновено наборът от данни трябва да съдържа само input и output, без instruction. Също така наборът от данни трябва да бъде двупосочен, което значи да има един пример, при който input е обяснение на задачата, а output е отговор на задачата и обратен пример при който input е отговор на задачата, а output е обяснение на задачата.

При инструкционната фина настройка за разбиране на информация, наборът от данни в .jsonl има следната структура:

```
{"instruction": "", "input": "Допълнение/вход", "output": "Отговор/изход."}
```

**//При тази фина настройка има само input и output, които са двупосочни и могат да разменят местата си.**

//Когато няма инструкции в тези примери, езиковият модел е принуден да мисли.

**Принцип за създаване на набор от данни от .docx към .jsonl за разбиране:** За всеки heading (заглавие) в .docx файл, python скрипт създава 4 примера + допълнения, които показват различни двупосочни връзки между заглавието и съдържанието под heading, като всички се записват в един и същ .jsonl файл. Логика на примерите:

Пример 1: Маркираният текст под заглавие се използва като изход, а немаркираният текст под заглавие се ползва като вход

```
{"instruction": "", "input": "Немаркиран текст", "output": "Маркиран текст"}
```

Пример 2: Маркираният текст под заглавие се използва като вход, а немаркираният текст под заглавие се ползва като изход.

```
{"instruction": "", "input": "Маркиран текст", "output": "Немаркиран текст"}
```

Пример 3: Целият текст под заглавие се използва като изход, а заглавието като вход

```
{"instruction": "", "input": "Заглавие", "output": "Маркиран и немаркиран текст под заглавие"}
```

Пример 4: Целият текст под заглавие се използва като вход, а заглавието като изход

```
{"instruction": "", "input": "Маркиран и немаркиран текст под заглавие ", "output": "Заглавие"}
}
```

Допълнителни примери: Ако под заглавието няма текст или маркиран текст, се създават само 2 примера за подзаглавия:

Пример: {"instruction": "", "input": "Заглавие", "output": "Имената на всичките подзаглавия"}

Пример: {"instruction": "", "input": "Имената на всичките подзаглавия", "output": "Заглавие"}

Винаги instruction остава празен.

//При фина настройка за разбиране на информация, input и output могат да бъдат ползвани също и по следните начини:

1. За разбиране на причинно-следствени връзки:

Пример 1: Input = причина 1, output = следствие 1

Пример 2: Input = следствие 1, output = причина 1

2. За разбиране на прилики

Пример 1: Input = описание на нещо, output = основни прилики на това нещо с други неща

Пример 2: Input = основни прилики на това нещо с други неща, output = описание на нещо

3. За разбиране на разлики

Пример 1: Input = описание на нещо, output = основни разлики на това нещо с други неща

Пример 2: Input = основни разлики на това нещо с други неща, output = описание на нещо

4. За разбиране на взаимодействия чрез аналогии

Пример 1: Input = описание на взаимодействие, output = аналогия за взаимодействието

Пример 2: Input = аналогия за взаимодействието, output = описание на взаимодействие

5. За създаване на асоциации и графика на знанието (knowledge graph): (//с тези трябва да се внимава, за да се избегне объркване/размиване, но работи)

Пример 1: Input = име на обект, output = имена на съставните части на този обект

Пример 2: Input = имена на съставните части на обект, output = име на този обект

6. За създаване на очаквания чрез условности: (//тези са задължителни; най-ефективни са)

Пример 1: Input = Ако стане това, output = Обикновено се случва следното нещо

Пример 2: Input = Обикновено се случва следното нещо, output = ако стане това

//Създаването на очаквания чрез условности става на принципа на conditionals (условните наклонения) от граматиката в английския език.

6.1. Zero Conditional – (Нулево условно наклонение)

Цел: да се установят ясни правила или закономерности. Ползва се за да може езиковият модел, да разбере, че резултатът е сигурен и неизбежен, и да се зададе рамка за поведение.

6.2. First Conditional – (Първо условно наклонение)

Цел: да се мотивира или насочи към конкретно действие. Ползва се, за да може езиковият модел да създаде очакване за бъдещ резултат и да се стимулира решителност.

6.3. Second Conditional – (Второ условно наклонение)

Цел: да се стимулира въображение и размисъл. Ползва се, за да може езиковият модел, да си представя алтернативи, възможности и потенциални сценарии.

#### 6.4. Third Conditional – (Трето условно наклонение)

Цел: да се анализират минали действия и да се извлекат поуки. Ползва се, за да може езиковият модел да осъзнава последствията от решенията и грешките и да се учи от опита си.

#### 6.5 Mixed Conditionals – (Смесени условни наклонения)

Цел: да се демонстрира връзката между минали действия и настоящи резултати. Използва се, за да се развие дълбока перспектива и разбиране на причинно-следствени зависимости.

#### 7. За оценяване на знания с вярно/грешно

Пример 1: Input = Твърдение или въпрос, output = вярно или грешно

Пример 2: Input = вярно или грешно, output = Твърдение или въпрос

#### 8. За оценяване на умения с правилно/неправилно

Пример 1: Input = умение/действие, output = правилно или неправилно

Пример 2: Input = правилно или неправилно, output = умение/действие

### 3. Фаза на работа с информация „2“ (за действия):

След като езиковият модел разбере научената информация, при необходимост може да я приложи на практика. Основният начин за работа с информация, е чрез комбиниране на учене с разбиране, тоест комбиниране на знанията и уменията, за да може моделът да генерира отговор. Тоест обикновено наборът от данни трябва да съдържа instruction, input, output. Също така наборът от данни трябва да бъде двупосочен, като примерите да имат следната структура:

(a = Instruction; b = Input; c = Output)

=> (a = Заглавие; b = Маркиран текст; c = Немаркиран текст)

//1. a + b = c

//2. a + c = b

//3. b + c = a

//4. c + b = a

При инструкционна фина настройка, когато има примери с instruction, input, output, обикновено instruction + input, се приемат за „prompt“, а output се приема за „completion“. При системата 4-4-2: за учене (learn), се създава набор от данни с 4 основни примера и при нужда 2 допълнителни (за всеки heading/заглавие); за разбиране (understand) се създава набор от данни с 4 основни примера и при нужда 2 допълнителни (за всеки heading/заглавие); за работа (work), се създава набор от данни с 2 основни примера и 2 допълнителни, които са спомагателни и се извличат от разбирането. Поради тази причина спомагателните примери не се броят за 2 броя и примерите за работа се приемат за 2 броя.

При (a, b, c) и (a, c, b), заглавието винаги е част от подканата (prompt), а при (b, c, a) и (c, b, a), заглавието винаги е част от отговора (completion). По този начин езиковият модел учи как от задача с условия, да генерира решение и как от решение (ситуация), да генерира задача и условия. Казано по-просто, по този начин езиковият модел се учи да мисли и работи индуктивно и дедуктивно. Моделът учи индуктивно, защото от частни положения трябва да изведе общи заключения. Моделът учи дедуктивно, защото от общи положения трябва да изведе частни заключения.

Методологията обхваща всички ключови нива на интелигентност: знание, разбиране и действие. Тя осигурява последователно развитие на езиковите модели, като натрупва информация, структурира я чрез логическо мислене и накрая я прилага на практика, което

дава дълбочина и гъвкавост. Двупосочността на данните повишава обобщаващата способност(генерализацията) на модела и неговата адаптивност към нови ситуации. Като цяло подходът е ефективен, защото не учи само на конкретни примери, а изгражда цялостно „мислене“, което е стабилно и приложимо. Повечето езикови модели се обучават на еднопосочни набори от данни, поради което са силни в индукцията, но са слаби в дедукцията, затова в тази методология са включени двупосочни примери. При индуктивния метод, езиковият модел се учи да открива закономерности, да разбира логика и намерение и да разпознава шаблони, докато при дедуктивния метод, езиковият модел се учи да прилага правила и да мисли структурно. Тоест при индуктивния метод гледната точка е на наблюдател, който може да описва, а при дедуктивния метод гледната точка е на извършител, който може да работи и мисли.

При инструкционна фина настройка за работа с информация, наборът от данни в .jsonl има следната структура:

```
{"instruction": "Въпрос", "input": "Допълнение/вход", "output": "Отговор/изход."}
```

//При тази фина настройка има instruction, input и output, които могат да разменят местата си.

**Принцип за създаване на набор от данни от .docx към .jsonl за работа:** За всеки heading (заглавие) в .docx файл, python скрипт създава 2 основни примера + 2 допълнителни, които идват от разбирането, които показват различни двупосочни връзки между заглавието и съдържанието под heading, като всички се записват в един и същ .jsonl файл. Логика на примерите:

Пример 1: Заглавието се ползва като инструкция, маркираният текст под заглавие се използва като изход, а немаркираният текст под заглавие се ползва като вход

```
{"instruction": "Заглавие", "input": "Немаркиран текст", "output": "Маркиран текст"}
```

Пример 2: Заглавието се ползва като инструкция, маркираният текст под заглавие се използва като вход, а немаркираният текст под заглавие се ползва като изход.

```
{"instruction": "Заглавие", "input": "Маркиран текст", "output": "Немаркиран текст"}
```

Пример 3: Заглавието се ползва като изход, маркираният текст под заглавие се използва като вход, а немаркираният текст под заглавие се ползва като инструкция

```
{"instruction": "Немаркиран текст", "input": "Маркиран текст", "output": "Заглавие"}
```

Пример 4: Заглавието се ползва като изход, маркираният текст под заглавие се използва като инструкция, а немаркираният текст под заглавие се ползва като вход.

```
{"instruction": "Маркиран текст", "input": "Немаркиран текст", "output": "Заглавие"}
```

Ако под заглавието няма текст или маркиран текст, не се генерират други допълнителни примери за него.

### **Обобщение за наборите от данни:**

Обиикновено от правилно структуриран .docx файл (или файлове), за всеки heading(или друг обект) могат да се извлекат по 4 примера за учене, 4 примера за разбиране и 2 примера за работа, което прави общо минимум 10 примера. По този начин наличната информация се ползва няколко пъти по-ефективно от обикновена фина настройка.

#### **Разделяне на наборите от данни по роли при системата 4-4-2:**

Разделянето по роли е принцип, който прави системата 4-4-2 по-ефективна.

**Учене (Learning):** Тук моделът се запознава с информацията. Примери са instruction + input, без output. Целта е да се формират асоциации и да се фиксира знанието без генериране на отговор. Това е „защитната линия“ на знанието. Не е за активно генериране на отговори, а за фиксиране и защита на информацията, така че моделът да я „запомни“ правилно и да я използва по-късно.

**Разбиране (Understanding):** Тук моделът изгражда логически връзки, причинно-следствени зависимости, аналогии и условни наклонения. Примери са input + output, без instruction. Броят примери за разбиране е по-голям, създавайки централна зона, която концентрира обработката на информацията. Това е „средната линия“ на логиката.

**Работа (Working):** Тук моделът комбинира наученото и разбраното, за да генерира резултати или решения. Примери са instruction + input + output. Малък брой примери, които са решаващи. Това е „нападателната линия“ на системата.

//Разделянето на данните по този начин напомня за „Таксономията на Блум“, която представлява система за подреждане на мисловни умения при учене.

#### **4. Двупосочност на примерите в наборите от данни при системата 4-4-2:**

Двупосочността позволява на модела да използва една и съща връзка между елементи по два начина. Например веднъж като input - output и веднъж като output - input.

Вместо да се добавят нови примери или да се повишава броя на параметрите, моделът повтаря същите знания в различен контекст. Това му позволява да разбира взаимовръзки по-дълбоко, да прилага знанията гъвкаво в нови ситуации и да реагира правилно, дори когато информацията е представена по различен начин.

С други думи, двупосочните примери удвояват полезността на всяка връзка, без да повишават размера на модела. Това е начин за качествено повишаване на ефективността, при което езиковият модел се държи като по-голям, въпреки че параметрите му остават същите след фината настройка.

#### **5. Влияние върху параметрите на езиковите модели – количествено повишаване**

Фината настройка не променя количеството параметри на езиков модел. При обикновените езикови модели „количествено повишаване на параметрите“ означава просто да добавиш повече тегла и слоеве към модела. Например, ако моделът има 2 милиарда параметъра, повишаване на капацитета се постига чрез добавяне на допълнителни слоеве или повишаване на размерите на векторите на токените. Това прави модела по-голям и позволява запомняне на повече примери и обработване на по-сложни зависимости, но при всяко количествено повишаване е нужно да се извърши обучение от нулата, а не фина настройка.

Това повишаване води до по-големи изчислителни разходи, повече памет и по-дълго време за обучение и в същото време количественото разширяване не гарантира по-добро разбиране на информацията или по-ефективно използване на вече наличните параметри. Тоест моделът просто може да стане по-голям и по-тежък, без гаранция за по-дълбоко или по-ефективно мислене.

//Аналогия: Ако приемем един езиков модел за електричен ток, количественото повишаване на параметрите на езиков модел наподобява повишаване на силата на тока (ампеража) и понижаване на напрежението му. При преноса на електрическа енергия, е по-ефективно ползването на ток с високо напрежение и по-малка сила на тока (ампераж).

Взаимодействията които извършват параметрите на езиковите модели, може да се каже, че представляват пренос на енергия, което значи че не е правилно да се работи в посока на повишаване на броя параметри, защото това води до загуби.

//Електричеството с по-голяма сила на тока (ампераж), съответно и езиковите модели с повече параметри, изискват по-мощен хардуер за функциониране. Единственият начин да бъде създаден по-добър езиков модел с по-малко ресурси, е като се заложи на качество, а не на количество.

## **6. Влияние върху параметрите на езиковите модели при системата 4-4-2 – качествено повишаване**

Системата 4-4-2 не повишава броя на параметрите, а повишава тяхната функционална ефективност чрез системно и структурирано обучение. Вместо да се учат само конкретни отговори, параметрите се обучават да изграждат многопосочни и обратими връзки между понятия, условия и действия. По този начин един параметър може да притежава много различни функции, което го прави по-умен, адаптивен и стабилен. По-точно казано, качествено повишаване на параметри при системата 4-4-2, е предназначено за повишаване на броя възможни взаимодействия(връзки), в които един параметър може да участва с други параметри. Единица параметър на обикновен модел също може да участва в повече от едно взаимодействие, но не в такъв мащаб като езиков модел, който е настроен по системата 4-4-2.

//Освен че наборите от данни са изградени по системата 4-4-2, самата фина настройка също се извършва по тази система. Това става като в една епоха на обучение, езиковият модел първо преминава през примерите(данните) за учене, после за разбиране и накрая за работа. Примерите не трябва да бъдат разбъркани.

Принципът на фината настройка се базира на разделяне на обучението на три последователни фази: учене, разбиране и действие. При учене, моделът формира стабилни асоциации между понятията чрез двупосочни примери, като output остава празен и няма натиск за генерация на отговор. Това позволява параметрите да се активират за изграждане на вътрешни връзки и да се натрупва знание без повърхностно запамятване. При разбиране, моделът ползва примери без instruction, като параметрите се насочват към изграждане на логически и обратими връзки, аналогии, причинно-следствени зависимости и условности. Така същите единици започват да участват в множество когнитивни функции, а не само в асоциативни връзки. При работа, примерите с instruction, input и output позволяват параметрите да прилагат наученото, да комбинират знанията и да извличат нови резултати, като същите единици вече изпълняват множество роли едновременно.

Ключовият механизъм е двупосочността, при която всяка информационна единица може да се използва в обратни и многопосочни връзки, което прави параметрите многопластови. Умишлено празните полета (instruction или output) при различните фази осигуряват, че параметрите се учат да изграждат структура и връзки, а не просто да възпроизвеждат отговори.

В резултат на това параметрите могат да се повишават качествено, което значи че един параметър може да участва в стотици или хиляди пъти повече взаимодействия, отколкото обикновен модел. Това води до значително по-голяма генерализация, по-малко „халюцинации“, по-добра стабилност при промяна на контекста и бърза адаптация към нови инструкции.

Това означава, че качествено повишаване на параметрите не се отнася за промяна на брой параметри. Качествено повишаване на параметрите се отнася за промяна на брой

взаимодействия („синаптични връзки“), които параметрите могат да извършват едновременно.

Повечето големи компании и разработчици ползват принципа на количественото повишаване, при който „Количествените изменения водят до качествени натрупвания“. Обаче основният начин, за конкуриране с „Голиат“ и за постигане на по-добър резултат с по-малко средства и усилия е, да се работи на принципа на качествено повишаване, при който ***„Качествените изменения водят до количествени натрупвания“***.

//Аналогия: Ако приемем езиков модел за електричен ток, качествено повишаване на параметрите на езиков модел наподобява понижаване на силата на тока (ампеража) и повишаване на напрежението му.

При качествено повишаване на параметри: Повишаването на напрежението на тока (на езиковия модел), се отнася за повишаване на броя възможни взаимодействия („синаптичните връзки“), които параметрите могат да извършат. Като повишаването на напрежението дава на зарядите повече енергия за движение, така се дава и на параметрите повече информация за взаимодействие.

При количественото повишаване на параметри: Понижаването на напрежението на тока (на езиковия модел), се отнася за понижаване на броя възможни взаимодействия („синаптичните връзки“), които параметрите могат да извършат. Като понижаването на напрежението дава на зарядите по-малко енергия за движение, така се дава и на параметрите по-малко информация за взаимодействие.

//Това значи, че способността за движение е първична форма на интелигентност в природата. Колкото по-голяма е способността за извършване на взаимодействия, толкова по-голяма е тяхната интелигентност.

//По-малко учене + повече разбиране води до по-добро генерализиране.

//Повече учене + по-малко разбиране води до по-лошо генерализиране (зубрене, overfitting), тоест неприложимост на информацията.

В този ред на мисли, е важно да се отбележи, че без значение дали се извършва нормална инструкционна фина настройка или такава по системата 4-4-2, и в двата случая видът на данните е от ключово значение, защото ако се ползват неподходящи данни дори и най-добрите методи за фина настройка няма да работят добре.

За да бъде извлечена максимална полза от инструкционната фина настройка по системата 4-4-2, е необходимо да се ползва набор от данни, който съдържа примери за дедуктивен детерминизъм. Причината за това е, че ако са познати и описани общи правила (причини), ще бъде логически възможно да се дедуцират (предвидят) относително точни резултати (следствия).

За извършването на инструкционна фина настройка по системата 4-4-2 при Olachra, се ползва набор от данни, който е базиран на четири двойки алгоритмични системи. Те съдържат информация за универсалните правила на приемане и предаване на сигнали в природата, което ги прави приложими в различни сфери на познанието. Въпреки тази система значително подобрява логиката на езиковия модел, тя не подобрява знанията на модела пряко, но ги подобрява косвено като създава нови знания (информация), на които моделът не е бил обучен първоначално.

//Това което забелязвам по време на работа е, че: „Интелигентността не се характеризира с количеството знания. Интелигентността се характеризира със способността за създаване на нови знания чрез разпознаване на модели (взаимодействия).“ Колкото по-голяма е тази способност, толкова повече се повишава адаптивността. Това напълня за твърдението на Чарлз Дарвин, че: „Не оцелява най-силният, нито най-интелигентният, а този който е най-адаптивен към промените.“

## II. Olachra - Инструкционна фина настройка по системата 4-4-2

### 1. Скрипт за инструкционна фина настройка чрез квантувана ниско-рангова адаптация (QLoRA) по системата 4-4-2 за Jupyter Lab:

```
# In[ ]:
```

```
# 1. Preparation for instruction fine-tuning
```

```
import os, json, torch, gc
from torch.utils.data import Dataset, SequentialSampler
from transformers import (
    AutoTokenizer, AutoModelForCausalLM, Trainer,
    TrainingArguments, DataCollatorForLanguageModeling, BitsAndBytesConfig
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
```

```
def clear_mem():
```

```
    gc.collect()
```

```
    torch.cuda.empty_cache()
```

```
clear_mem() # Clears VRAM before you start
```

```
print(f"  GPU: {torch.cuda.get_device_name(0)}")
```

```
# In[ ]:
```

```
# 2. Configuration for instruction fine-tuning
```

```
MODEL_PATH = "/workspace/work/LLM" # Path for the base language model for fine-tuning
                                         (contains all the .safetensors files, tokenizers, configs and etc.)
```

```
DATASET_PATH = "/workspace/work/datasets/cognitive_package_magixtral.jsonl" # Path for the
dataset for fine-tuning
```

```
OUTPUT_DIR = "/workspace/work/outputs/MAGIXTRAL_7B" # Path for the checkpoints created
during fine-tuning and for the LoRA adapters at the end.
```

```
MAX_LENGTH = 1024 # The number of tokens (~800 words per example from the dataset). If
there are bigger examples than that, the model will not learn the whole example.
```

```
# Increasing max_length, leads to more VRAM usage and vice versa. You can set max_length lower
than your dataset example length, only if you add chunking logic, but here that's not the case.
```

```
BATCH_SIZE = 1 # The number of examples, the language model looks at, at the same time.
```

`GRAD_ACC = 16` # The number of batches, the language model adds up before updating its weights.  
 # `Batch_size * grad_acc = training step` (the number of examples the language model see, before updating its weights)  
 # Increasing `batch_size`, leads to more VRAM usage and vice versa.  
 # Increasing `gradient_accumulation`, leads to the increase of the training time.  
 # If you haven't got enough VRAM, you can compensate with increasing the time for fine-tuning (in case you need to make the model see more examples at once).  
 # In this case, 1 training step will be 16 examples.

`EPOCHS = 1` # The number of times, the language model runs through the examples from the dataset.  
 # Repetition is mother of knowledge when it occurs during learning (in the dataset) and father of dulling when it occurs during comprehension/understanding (with more epochs of fine-tuning).

`LR = 5e-5` # Learning Rate is the size of the step, the language model make before updating its weights. (in this case, `5e-5` is scientific notation for the number 0.00005)  
 # Bigger steps for fast, but unstable learning.  
 # Smaller steps for slow, but relatively stable learning.

`LORA_R = 128` # rank of the Low-Rank Adaptation (LoRA rank).  
 # Increasing `lora_r`, leads to more VRAM usage and vice versa. When you increase `lora_r`, you increase the number of parameters that will be fine-tuned.  
 # During fine-tuning, I noticed that when you increase `lora_r`, you make the language model think more analytically.  
 # So increasing of `lora_r` makes the language model sharper, while decreasing the `lora_r` makes the model less adaptive.

`LORA_ALPHA = 4` # scaling factor for the LoRA update.  
 # Changing `lora_alpha`, leads to changing the influence, that the dataset cause to the parameters that will be fine-tuned  
 # During fine-tuning, I noticed that when you decrease `lora_alpha`, you make the language model think more critically.  
 # So increasing of `lora_alpha` makes the language model more naive (trusts easy to the new data from the dataset), while decreasing the `lora_alpha` makes the model more aware (trusts hard and needs time).  
 # `Lora_r` is like "worldview" (мυρογλεδ) - The bigger it is, the bigger is the capacity for learning.  
 # `Lora_alpha` is like "trust" (δoβεpue) - The smaller it is, the bigger is the understanding.

`LORA_DROPOUT = 0.05` # This causes the language model to ignore 5 % of the trained parameters, which leads to not becoming so attached to the new information in the dataset.  
 # This is the humility (cмyпенyемо) of the language model. The bigger it is, the better it learns.  
 # The bigger the worldview and the humility and the smaller the trust, leads to fine-tuning better language models.

`# In[ ]:`

*# 3. Checking if there are language model and dataset in the paths*

*def check\_paths():*

*m\_ok = os.path.exists(MODEL\_PATH)*

*d\_ok = os.path.exists(DATASET\_PATH)*

*print(f"LLM: {'✅' if m\_ok else '❌'})"*

*print(f"dataset: {'✅' if d\_ok else '❌'})"*

*if not (m\_ok and d\_ok):*

*raise FileNotFoundError("Error: Check file paths!")*

*check\_paths()*

*# In [ ]:*

*# 4. QLoRA configuration and loading the language model*

*tokenizer = AutoTokenizer.from\_pretrained(MODEL\_PATH, use\_fast=True)*

*tokenizer.pad\_token = tokenizer.eos\_token*

*tokenizer.padding\_side = "right"*

*bnb\_config = BitsAndBytesConfig(*

*load\_in\_4bit=True,*

*bnb\_4bit\_use\_double\_quant=True,*

*bnb\_4bit\_quant\_type="nf4",*

*bnb\_4bit\_compute\_dtype=torch.float16*

*)*

*model = AutoModelForCausalLM.from\_pretrained(*

*MODEL\_PATH,*

*quantization\_config=bnb\_config,*

*device\_map="auto",*

*trust\_remote\_code=True,*

*low\_cpu\_mem\_usage=True*

*)*

*# Enable memory saving*

*model.gradient\_checkpointing\_enable()*

*model = prepare\_model\_for\_kbit\_training(model)*

*lora\_config = LoraConfig(*

*r=LORA\_R,*

*lora\_alpha=LORA\_ALPHA,*

*target\_modules=["q\_proj", "k\_proj", "v\_proj", "o\_proj", "gate\_proj", "up\_proj", "down\_proj"],*

*lora\_dropout=LORA\_DROPOUT,*

*bias="none",*

*task\_type="CAUSAL\_LM"*

*)*

*model = get\_peft\_model(model, lora\_config)*

```

model.print_trainable_parameters()

# In[ ]:

# 5. Phases during fine-tuning
# The language model goes through the "learn" examples first, then the "understand" examples,
and finally the "work" examples.

class CognitiveSequentialDataset(Dataset):
    def __init__(self, path, tokenizer, max_length):
        self.tokenizer = tokenizer
        self.max_length = max_length
        with open(path, "r", encoding="utf-8") as f:
            self.examples = [json.loads(line) for line in f if line.strip()]

    def __len__(self):
        return len(self.examples)

    def __getitem__(self, idx):
        ex = self.examples[idx]
        phase = ex.get("phase", "work") # default
        instr = ex.get("instruction", "").strip()
        inp = ex.get("input", "").strip()
        out = ex.get("output", "").strip()

        # Format
        prompt_part = f"#### Instruction:\n{instr}\n\n#### Input:\n{inp}\n\n#### Response:\n"
        full_text = prompt_part + out + self.tokenizer.eos_token

        # Tokenization
        enc = self.tokenizer(full_text, truncation=True, max_length=self.max_length,
padding=False)
        input_ids = list(enc["input_ids"])
        labels = list(input_ids)

        # MASKING: If it is not a 'learn' phase, the prompt is masked
        if phase != "learn":
            prompt_enc = self.tokenizer(prompt_part, add_special_tokens=False, truncation=True,
max_length=self.max_length)
            prompt_len = len(prompt_enc["input_ids"])
            for i in range(min(prompt_len, len(labels))):
                labels[i] = -100

        return {
            "input_ids": torch.tensor(input_ids, dtype=torch.long),
            "attention_mask": torch.tensor(enc["attention_mask"], dtype=torch.long),
            "labels": torch.tensor(labels, dtype=torch.long)

```

```

    }

train_dataset = CognitiveSequentialDataset(DATASET_PATH, tokenizer, MAX_LENGTH)

# In[ ]:

# 6. Giving the examples from the dataset sequentially to the language model (shuffle=False)
class StrictSequentialTrainer(Trainer):
    def _get_train_sampler(self, dataset):
        return SequentialSampler(dataset)

# In[ ]:

# 7. Final settings
SAVE_STEPS = 250      # Saves checkpoint for the fine-tuning on every 250 training steps. (250
* 16 = 4000 examples)
SAVE_TOTAL_LIMIT = 2  # Stores only the last two checkpoint files to save disk space. (The
bigger the lora_r, the bigger the checkpoints and adapters get).

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=BATCH_SIZE,
    gradient_accumulation_steps=GRAD_ACC,
    learning_rate=LR,
    num_train_epochs=EPOCHS,

    # Optimizations
    fp16=True, # Makes the language model use two times less VRAM during calculations. Faster
fine-tuning.
    optim="paged_adamw_8bit", # Optimizer for the weight updates
    weight_decay=0.1,      # Help against overtraining (overfitting)

    # Logging (omnem)
    logging_steps=10,      # Fine-tuning progress bar updates on every 10 training steps (10 steps
* 16 examples = 160 examples)
    report_to="none",

    # Saving checkpoints using steps
    save_strategy="steps",
    save_steps=SAVE_STEPS,
    save_total_limit=SAVE_TOTAL_LIMIT,

    # Saving memory and stabilizing the fine-tuning
    gradient_checkpointing=True,
    gradient_checkpointing_kwargs={"use_reentrant": False},

    # Optional for Windows

```

```

    dataloader_pin_memory=True, # Load examples in a reserved space in the RAM to get them
ready for the GPU when needed. Reducing the time, the GPU stays empty while waiting the CPU
to transfer the training examples.
    dataloader_num_workers=0    # Number of parallel loading processes. In this case is only the
main one.
)

trainer = StrictSequentialTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
)

clear_mem()
print(f"Instruction Fine-Tuning (IFT) via Quantized Low-Rank Adaptation (QLoRA) with the
System 4-4-2 (inside {OUTPUT_DIR})...")

# In[ ]:

# 8. Beginning of the language model fine-tuning
print("Start")
trainer.train()

# In[ ]:

# 9. Ending of the language model fine-tuning
final_path = os.path.join(OUTPUT_DIR, "cognitive_final_model") # Saves the LoRA adapters
into the output folder
trainer.save_model(final_path)
tokenizer.save_pretrained(final_path)
print(f"End! The LoRA adapters of the language model are saved in: {final_path}")

```

//В този случай се ползва `shuffle=False`, което значи че моделът ще учи примерите в реда, в който са подредени в `.jsonl` файла, тоест първо за учене, после за разбиране и накрая за работа.

//Възможно е да се ползва `shuffle=True`, но за целта са нужни три отделни файла, също и промяна на кода така, че моделът да преминава последователно през фазите файл по файл, като примерите от всеки набор от данни ще бъдат разбъркани.

//При обикновена инструкционна фина настройка, обикновено кривата на „training loss“ е постоянна и се понижава равномерно. Тя относително представлява права. При инструкционна фина настройка по системата 4-4-2, кривата на „training loss“ е променлива и се понижава неравномерно. Тя относително представлява синусоида.

//При този вид фина настройка трябва да има само една епоха на обучение, при която `shuffle=False`, но може да се имитира `shuffle=True`, чрез „ротация“ на фазите на обучение:

*EPOCHS = 1 # The number of times, the language model runs through the examples from the dataset.*

*# Repetition is mother of knowledge when it occurs during learning (in the dataset) and father of dulling when it occurs during comprehension/understanding (with more epochs of fine-tuning).*

Обикновено една епоха на обучение не е достатъчна при тази конфигурация. В същото време не може да се ползва повече от една епоха, защото това ще доведе до претоварване на същите обучавани параметри.

#### **//Обяснение:**

Когато извършваш фина настройка чрез квантувана ниско-рангова адаптация, обучението се извършва само върху малък процент от общия брой параметри на езиковия модел. В този случай, наборът от данни има определена последователност:

(a = примери за учене, b = примери за разбиране и c = примери за работа).

Когато обучаваш езиковия модел върху набор от данни, без да ползваш разбъркване (shuffle = True), ако има повече от една епоха на обучение, примерите от набора от данни ще променят теглата на обучаваните параметри по същия начин, а това не се отразява добре на работата на езиковия модел (влошава го).

Решението в този случай, е да се ползва една ротационна епоха на обучение (3 в 1), която се равнява на три епохи, като това става по следния начин:

Наборът от данни се създава по такъв начин, че примерите да са в следната последователност:

-Епоха на обучение 1: a, b, c (учене, разбиране, работа);

-Епоха на обучение 2: b, c, a (разбиране, работа, учене);

-Епоха на обучение 3: c, a, b (работа, учене, разбиране);

//Това става чрез скрипта за съединяване в Midextral.

//Само по горепосочения начин, примерите преминават през трите групи обучавани параметри на езиковия модел, като се припокриват минимално, заради различното съотношение на примерите (като брой и процент).

#### **- Работа след приключване на инструкционната фина настройка на езиковия модел:**

1. След приключване на инструкционна фина настройка, е необходимо да се извърши съединяване (merge) на LoRA адаптерите с базовия езиков модел, върху който е проведено обучението. По този начин се получава крайният обучен модел.

2. След съединяването е необходимо, да се извърши конвертиране на езиковия модел от формат .safetensors във формат .gguf, който е оптимизиран за локално изпълнение и по-ефективна работа.

3. След конвертирането е необходимо, да се извърши квантуване на езиковия модел, с цел понижаване на размера, оптимизиране на производителността и по-ниска консумация на ресурси.

4. След извършване на квантуване, езиковият модел може да бъде ползван и тестван, като за целта е необходимо, да се извършат допълнителни настройки за подобряване на качеството на отговорите при генериране.

### III. Olachra - Система за ползване на големи езикови модели - „работа/действие“

За ползване на обученения модел може да се ползва само софтуер като „Ollama“. В случая обаче, езиковият модел работи в специално разработена система, която е комбинация от „Ollama“, „LangChain“ и „ChromaDB“ (Olachra engine).

Наименованието на системата на български език е „Олахра“, а на английски език е „Olachra“ от Ollama, LangChain, ChromaDB.

Наименованието съдържа в себе си думите (Ola и Ra). Ola разговорно значи „Здравей“ на испански език. Ra символично значи „светлина/просвета“. Тоест наименованието „Олахра“ буквално значи: „Здравей светлина/просвета“ – „Hello light/enlightenment“, което е аналогично на израза „Здравей свят“ - „Hello world“ при програмирането.

//В буквален превод от испански език Ola значи „вълна“, а Hola значи „Здравей“.

#### Общи сведения:

-Създаване на способност за учене - памет за знания: Чрез комбиниране на езика за програмиране „Python“ и библиотеката „ChromaDB“ с „Ollama“, се създава декларитивната памет на системата. Чрез нея езиковият модел може да съхранява знания и контекст(смисъл), да учи и да ползва натрупания с времето опит. Библиотеката „ChromaDB“ позволява да бъде търсена информация по контекст(съдържание, смисъл), а не по ключови думи от заглавия, текст и таблици, защото това е векторна база данни.

-Създаване на способност за разбиране - памет за умения: Чрез комбиниране на езика за програмиране „Python“ и библиотеката „LangChain“ с „Ollama“, се създава процедурната памет (логиката) на системата. Библиотеката позволява на езиковия модел да структурира и организира информация, да планира, да взема решения (не просто да прави изводи), в зависимост от режима на работа.

-Създаване на информационна система за работа на езиковия модел: Съставните части на системата може да се опишат образно по следния начин: „Ollama“ - тялото на системата, „LangChain“ - процедурната памет, „ChromaDB“ - декларитивната памет, „LLM“ (езиковият модел) – мозъкът.

#### **Опростен принцип на работа на системата при ръчен режим:**

1. Потребител въвежда подкана(prompt) в системата, след което тя се приема от „LangChain“;
2. „LangChain“ предава заявка към (пита) „ChromaDB“ за информация, ако сметне за нужно, като за целта се ползва езиков модел за векторизиране на данните;
3. „ChromaDB“ приема заявката и предава (връща) няколко текста(информация) във векторизиран вид;
4. „LangChain“ приема информацията и комбинира тези текстове (информация), след което ги предава към (праща на) „Ollama“ като финална подкана(prompt);
5. „Ollama“ приема информацията, след което генерира отговор въз основа на нея чрез езиковия модел;

#### **Принцип на работа на съставните части на системата:**

**1. „ChromaDB“ е „декларитивната памет“ на системата:** „ChromaDB“ е векторна база данни, в която се запазва „значението“ на текста(информацията), а не просто думите. Чрез нея се преобразува текст във вектор (числа, които представят смисъла), може да се търси по смисъл, а не по ключови думи и да се даде на модела „декларитивна памет“.

//Декларитивната памет е памет за знанията. Пример: „Знам какво е дадено нещо“.

2. **„LangChain“ е „процедурната памет“ на системата:** „LangChain“ е програмен инструмент (framework – основа за разработване на приложения и медиатор), който свързва езиковия модел с памет, данни и външни инструменти. Чрез него може да се създават структури като:

-Chain (верига) - последователност от стъпки (пример: “прочети текст → обобщи → преведи”).

-Memory (памет) - за запомняне на предишни разговори и добавяне в нов контекст.

-Agent (агент) - позволява на модела сам да решава какво действие да предприеме.

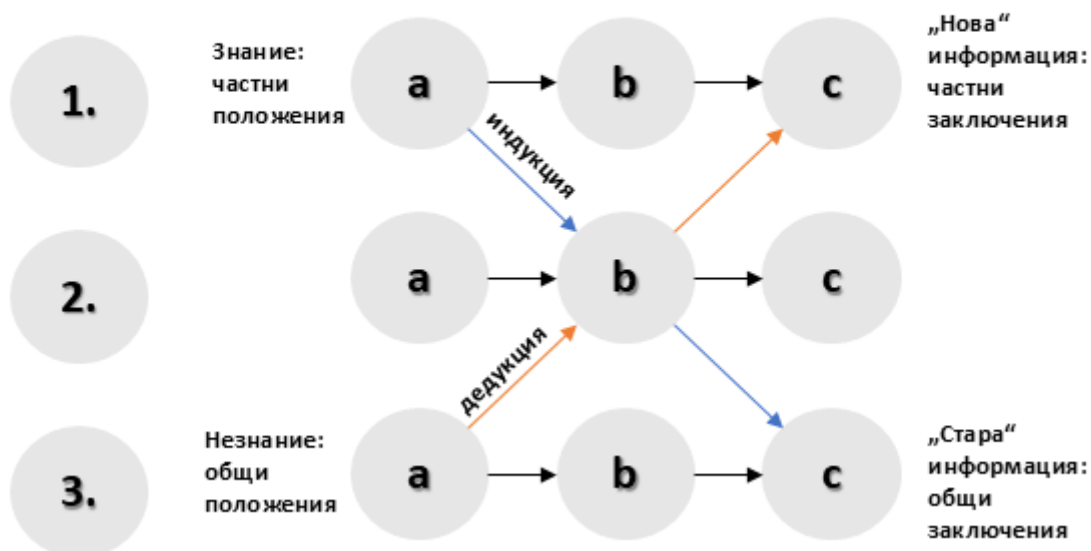
Позволява да се комбинират модели, файлове, бази данни, API-та и други източници на информация.

//Процедурната памет е памет за уменията. Пример: „Знам как да направя дадено нещо“.

3. **„Ollama“ е „тялото“ на системата:** „Ollama“ е програма, която позволява ползване на езикови модели локално на компютър – без интернет, без външни сървъри и т.н. Тя изпълнява изчисленията на езиковия модел (в процесора - CPU или във видеокартата -GPU). Приема твоя подкана/въпрос (prompt) и връща отговор (completion). Работи чрез локален API, поради което може да бъдат пращани заявки чрез „Python“.

//Пример: „Мога да направя дадено нещо“.

### Схема на концепция за работа на AGI – общ изкуствен интелект



#### Фази:

**a** = учене/знание;

**b** = разбиране/умение;

//Разбирането на научената информация по време на фина настройка, е от ключово значение, защото както се вижда на схемата горе, разбирането е пресечната точка между индукция и дедукция.

**c** = работа/действие;

*//,, Фокусът върху знанието води до развитие (индукция), а фокусът върху незнанието води до прогрес (дедукция)“. От знанията може да се извежда позната(стара) информация, а от незнанията може да се извежда непозната(нова) информация. Фокусът върху частни положения води до общи заключения (развитие), а фокусът върху общи положения води до частни заключения (прогрес).*

*-По метода на индукцията (обикновен изкуствен интелект), от частни положения се извеждат общи заключения от езиковия модел. При индукцията, фокусът е върху знанието, поради което моделът става по-ригиден. В този случай, моделът може да генерира информация, която е присъствала в набора от данни при обучението.*

*-По метода на дедукцията (общ изкуствен интелект), от общи положения се извеждат частни заключения от езиковия модел. При дедукцията, фокусът е върху незнанието, поради което моделът става по-адаптивен. В този случай, моделът може да генерира информация, която не е присъствала в набора от данни при обучението.*

*//Може да се каже, че обикновен изкуствен интелект, е всяка информационна система базирана на индукция, а общ изкуствен интелект, е всяка информационна система базирана на дедукция, която може да комбинира и двете.*

*//В операционната система "Magi", е изписан текст под логото: "powered by Olachra", защото се има в предвид, че се ползва комбинацията на Ollama + Langchain + ChromaDB, което в тази операционна система е като светата тройца, също така в бекенда е наличен "class OlachraEngine", който е от ключово значение.*

*//Системата 4-4-2 се наблюдава на микро и макро ниво в цялата екосистема. На микро ниво при взаимодействие във всяка отделна система (платформа) и на макро ниво при взаимодействие извън всяка система. На микро ниво при вътрешно взаимодействие във всяка отделна система. На макро ниво при външно взаимодействие между системите.*